

Numerical Analysis

Lecture Notes

Yaé Ulrich Gaba

Licence L3 — 2025–2026

Contents

List of Algorithms	vii
Preface	ix
1 Floating-Point Arithmetic, Errors and Stability	1
1.1 Motivating example	1
1.2 Representation of numbers in a computer	1
1.3 Rounding errors and propagation	2
1.4 Conditioning of a problem	2
1.5 Numerical stability	3
1.6 Implementation	3
1.7 Exercises	4
2 Root Finding	5
2.1 Motivating example	5
2.2 Bisection method	5
2.3 Newton's method	6
2.4 Secant method	7
2.5 Fixed-point method	7
2.6 Comparison of methods	7
2.7 Numerical example	7
2.8 Implementation	7
2.9 Exercises	9
3 Direct Methods for Linear Systems	11
3.1 Motivating example	11
3.2 Gaussian elimination and LU factorisation	11

3.3	Partial pivoting	12
3.4	Cholesky factorisation	12
3.5	Numerical example	13
3.6	Implementation	13
3.7	Exercises	14
4	Iterative Methods for Linear Systems	15
4.1	Motivating example	15
4.2	General principle	15
4.3	Jacobi method	16
4.4	Gauss–Seidel method	16
4.5	Successive over-relaxation (SOR)	16
4.6	Conjugate gradient method	16
4.7	Implementation	17
4.8	Exercises	18
5	Polynomial Interpolation	21
5.1	Motivating example	21
5.2	The interpolation problem	21
5.3	Lagrange form	21
5.4	Newton form	22
5.5	Interpolation error	22
5.6	Chebyshev nodes	23
5.7	Cubic splines	23
5.8	Implementation	23
5.9	Exercises	24
6	Approximation — Least Squares and Chebyshev	27
6.1	Introduction	27
6.2	Discrete Least Squares	28
6.2.1	Problem formulation	28
6.2.2	Normal equations	28
6.2.3	Polynomial least squares via QR factorization	29
6.3	Continuous Least Squares and Orthogonal Polynomials	29

6.3.1	Continuous least squares	29
6.3.2	Orthogonal polynomials	29
6.4	Chebyshev Polynomials	30
6.5	Best Uniform Approximation (Chebyshev / Minimax)	30
6.6	Near-best Approximation via Chebyshev Interpolation	31
6.7	Python Implementation	31
6.8	Exercises	33
7	Numerical Differentiation and Integration	35
7.1	Introduction	35
7.2	Numerical Differentiation	35
7.2.1	Finite difference formulas	35
7.2.2	Higher-order formulas	37
7.3	Numerical Integration (Quadrature)	37
7.3.1	The trapezoidal rule	37
7.3.2	Simpson's rule	38
7.4	Richardson Extrapolation	38
7.5	Convergence Study: $\int_0^1 e^{-x^2} dx$	39
7.6	Implementations	39
7.7	Exercises	41
8	Gaussian Quadrature	43
8.1	Introduction	43
8.2	Fundamental Theorem of Gaussian Quadrature	43
8.3	Gauss–Legendre Quadrature	44
8.3.1	Change of interval	45
8.4	Error Formula	45
8.5	Other Gaussian Quadrature Rules	45
8.5.1	Gauss–Laguerre	45
8.5.2	Gauss–Hermite	46
8.6	Composite Gauss Quadrature	46
8.7	Implementations	46
8.8	Exercises	48

9	Numerical Solution of Ordinary Differential Equations	51
9.1	Introduction	51
9.2	The Cauchy Problem	51
9.3	Euler's Methods	52
9.3.1	Explicit (forward) Euler method	52
9.3.2	Implicit (backward) Euler method	53
9.4	Runge–Kutta Methods	53
9.4.1	Classical fourth-order Runge–Kutta (RK4)	54
9.5	Stability Analysis	54
9.5.1	Stability region plots	55
9.6	Convergence Study: $y' = -y$	55
9.7	Implementations	56
9.8	Exercises	58
10	Numerical Computation of Eigenvalues and Eigenvectors	61
10.1	Introduction	61
10.2	Spectral Properties	61
10.3	Gerschgorin's Theorem	62
10.3.1	Gerschgorin discs visualization	63
10.4	Power Iteration	63
10.5	Inverse Iteration with Shift	64
10.6	The QR Algorithm	65
10.6.1	Basic QR algorithm	65
10.6.2	QR algorithm with Hessenberg reduction	65
10.6.3	QR algorithm with shifts	66
10.7	Singular Value Decomposition (SVD)	66
10.8	Convergence Study	67
10.9	Implementations	67
10.10	Exercises	71
	Formula Sheet	75
.1	Vector and Matrix Norms	75
.2	Errors	75
.3	Quadrature Rules	75
.4	ODE Methods	75

List of Algorithms

1	Bisection	5
2	Newton's method	6
3	LU factorisation (without pivoting)	12
4	Cholesky factorisation	13
5	Conjugate gradient	17
6	Explicit Euler method	52
7	Power iteration	63
8	Inverse iteration with shift	64

Preface

Numerical analysis is the art of solving mathematical problems *approximately but with controlled error*. Unlike theoretical analysis, which establishes existence and uniqueness of solutions, numerical analysis is concerned with *actually computing* them and *quantifying the error incurred*.

This course covers the fundamentals: floating-point arithmetic, solving linear systems, interpolation, numerical integration, ordinary differential equations, and eigenvalue problems. Each method is accompanied by its error analysis, pseudocode, and implementations in Python and Julia.

Prerequisites. Real Analysis I & II, linear algebra, basic programming.

References.

- QUARTERONI, Sacco, Saleri — *Numerical Mathematics*, Springer.
- SÜLI, Mayers — *An Introduction to Numerical Analysis*, Cambridge.
- TREFETHEN, Bau — *Numerical Linear Algebra*, SIAM.
- HIGHAM — *Accuracy and Stability of Numerical Algorithms*, SIAM.
- DEMAILLY — *Analyse Numérique et Équations Différentielles*, EDP Sciences.

Chapter 1

Floating-Point Arithmetic, Errors and Stability

“Numerical computation is the art of giving the right answer to a slightly different problem.”

On February 25, 1991, a Patriot missile battery in Dhahran, Saudi Arabia, fails to intercept an incoming Scud missile, which kills 28 American soldiers. The post-mortem identifies a single root cause: the battery’s internal clock, accumulating ticks over 100 hours of continuous operation, had drifted by 0.34 seconds because of the binary truncation of one tenth in the system’s floating-point representation. Numerical analysis is, in part, the discipline that prevents that drift. The IEEE 754 standard for floating-point arithmetic — finalized in 1985 — specifies exactly how a real number is approximated, how rounding occurs, and what error to expect. This chapter formalizes the floating-point representation, defines machine epsilon and unit roundoff, distinguishes forward and backward error, and discusses numerical stability: the property that makes an algorithm survive contact with finite precision.

1.1 Motivating example

Let us compute $f(x) = \frac{1-\cos x}{x^2}$ for $x = 10^{-8}$ in double precision. The theoretical result is ≈ 0.5 , but Python returns 0.0! The **catastrophic cancellation** between 1 and $\cos x$ eliminates all significant digits. Understanding floating-point arithmetic is essential.

1.2 Representation of numbers in a computer

Definition 1.1 (Floating-point number). In base β with t significant digits, a floating-point number is written:

$$x = \pm m \times \beta^e, \quad m = d_0.d_1d_2 \cdots d_{t-1}, \quad d_0 \neq 0,$$

where $e_{\min} \leq e \leq e_{\max}$ and $0 \leq d_i < \beta$.

Definition 1.2 (IEEE 754 Standard).

Format	Bits	Mantissa	$\varepsilon_{\text{mach}}$
Single precision	32	23+1 bits	$\approx 1.19 \times 10^{-7}$
Double precision	64	52+1 bits	$\approx 2.22 \times 10^{-16}$

Definition 1.3 (Machine epsilon). The **machine epsilon** $\varepsilon_{\text{mach}}$ is the smallest $\varepsilon > 0$ such that $\text{fl}(1 + \varepsilon) > 1$:

$$\varepsilon_{\text{mach}} = \frac{1}{2}\beta^{1-t}.$$

For every representable $x \in \mathbb{R}$, $\text{fl}(x) = x(1 + \delta)$ with $|\delta| \leq \varepsilon_{\text{mach}}$.

1.3 Rounding errors and propagation

Definition 1.4 (Absolute and relative error).

$$\text{Absolute error : } E_a = |x - \tilde{x}|, \quad (1.1)$$

$$\text{Relative error : } E_r = \frac{|x - \tilde{x}|}{|x|} \quad (x \neq 0). \quad (1.2)$$

Theorem 1.5 (Error propagation). If $\tilde{x} = x(1 + \varepsilon_x)$ and $\tilde{y} = y(1 + \varepsilon_y)$:

- $\tilde{x} \cdot \tilde{y} \approx xy(1 + \varepsilon_x + \varepsilon_y)$: relative errors add up.
- $\tilde{x} - \tilde{y} \approx (x - y) + x\varepsilon_x - y\varepsilon_y$: if $x \approx y$, the relative error blows up.

Warning

Catastrophic cancellation occurs when subtracting nearly equal numbers. Example: $1 - \cos x$ for small x . Remedy: use $2 \sin^2(x/2)$.

1.4 Conditioning of a problem

Definition 1.6 (Condition number). The **condition number** of a problem f at the point x is:

$$\kappa(f, x) = \left| \frac{xf'(x)}{f(x)} \right|.$$

A problem is **well-conditioned** if $\kappa \sim 1$, **ill-conditioned** if $\kappa \gg 1$.

Definition 1.7 (Matrix condition number). For $Ax = b$:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|.$$

The relative error on x is amplified by $\text{cond}(A)$:

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}.$$

1.5 Numerical stability

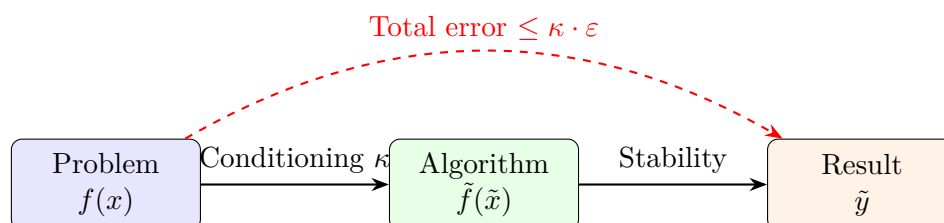
Definition 1.8 (Stable algorithm). An algorithm is **stable** if it gives the exact solution of a **slightly perturbed** problem. It is **backward stable** if the perturbation is of order $\varepsilon_{\text{mach}}$.

Method

The accuracy of the result depends on two factors:

1. The **conditioning** of the problem (inherent, cannot be changed).
2. The **stability** of the algorithm (the programmer's choice).

Rule: $\text{error} \lesssim \text{cond} \times \varepsilon_{\text{mach}}$.



1.6 Implementation

Python

```

import numpy as np

# Epsilon machine
eps = np.finfo(float).eps
print(f"eps_mach = {eps}") # 2.220446049250313e-16

# Cancellation catastrophique
x = 1e-8
f_bad = (1 - np.cos(x)) / x**2
f_good = 2 * np.sin(x/2)**2 / x**2
print(f"Mauvais : {f_bad}") # 0.0
print(f"Bon      : {f_good}") # 0.49999999...

# Conditionnement d'une matrice
A = np.array([[1, 1], [1, 1.0001]])
print(f"cond(A) = {np.linalg.cond(A):.1f}") # ~40000
  
```

Julia

```

# Epsilon machine
println("eps_mach = ", eps(Float64))

# Cancellation
x = 1e-8
f_bad = (1 - cos(x)) / x^2
f_good = 2sin(x/2)^2 / x^2
println("Mauvais : $f_bad")
println("Bon      : $f_good")

# Conditionnement
using LinearAlgebra
A = [1.0 1.0; 1.0 1.0001]
println("cond(A) = ", cond(A))

```

1.7 Exercises

Exercise 1.1 (★ – Machine epsilon). Write a program that computes $\varepsilon_{\text{mach}}$ by bisection. Compare with `np.finfo(float).eps`.

Exercise 1.2 (★ – Cancellation). Propose a stable computation of $\sqrt{x+1} - \sqrt{x}$ for large x . Verify numerically.

Exercise 1.3 (★★ – Condition number). Compute the condition number of the problem $f(x) = \ln x$ at the point $x = 1$. Interpret.

Exercise 1.4 (★★ – Hilbert matrix). Let H_n be the Hilbert matrix ($h_{ij} = 1/(i+j-1)$). Compute `cond( $H_n$ )` for $n = 2, \dots, 15$. What do you observe?

Exercise 1.5 (★★★ – Project). Study the numerical stability of evaluating the polynomial $p(x) = \sum a_i x^i$ using Horner's algorithm vs. naive evaluation. Compare relative errors for high-degree polynomials.

Key Formulas

$$\begin{aligned}
 \text{fl}(x) &= x(1 + \delta), \quad |\delta| \leq \varepsilon_{\text{mach}} & \varepsilon_{\text{mach}} &= \frac{1}{2}\beta^{1-t} \\
 \kappa(f, x) &= |xf'(x)/f(x)| & \text{cond}(A) &= \|A\| \cdot \|A^{-1}\|
 \end{aligned}$$

Chapter 2

Root Finding

In 1669, Newton publishes in *De analysi per aequationes numero terminorum infinitas* a method for computing the roots of polynomial equations using tangent lines: at each step, replace the curve by its tangent, solve the elementary linear equation, iterate. The method is so efficient that it converges quadratically — each iteration roughly doubles the number of correct decimals. Three centuries later, it remains the default algorithm for solving nonlinear equations in every piece of scientific software, from MATLAB to GPS receivers. But Newton did not say everything: the method can diverge, oscillate, or converge to the wrong root depending on the starting point. This chapter introduces the classical algorithms (bisection, secant, Newton, modified Newton, fixed-point iteration), analyzes their orders of convergence, and discusses the globalization strategies that make these methods reliable in practice.

2.1 Motivating example

Finding x such that $e^x = 3x$ amounts to finding a root of $f(x) = e^x - 3x$. No closed-form solution exists: a numerical method is needed.

2.2 Bisection method

Theorem 2.1 (Intermediate Value Theorem). *If $f \in C([a, b])$ and $f(a)f(b) < 0$, then $\exists c \in (a, b)$ such that $f(c) = 0$.*

Algorithm 1: Bisection

Input: f , a , b , tolerance ε

Output: Approximation of the root c

while $b - a > \varepsilon$ **do**

$c \leftarrow (a + b)/2$;

if $f(a) \cdot f(c) < 0$ **then**

$b \leftarrow c$

else

$a \leftarrow c$

return c

Theorem 2.2 (Convergence of bisection). *After n iterations: $|c_n - x^*| \leq \frac{b-a}{2^{n+1}}$. The convergence is **linear** with rate $1/2$.*

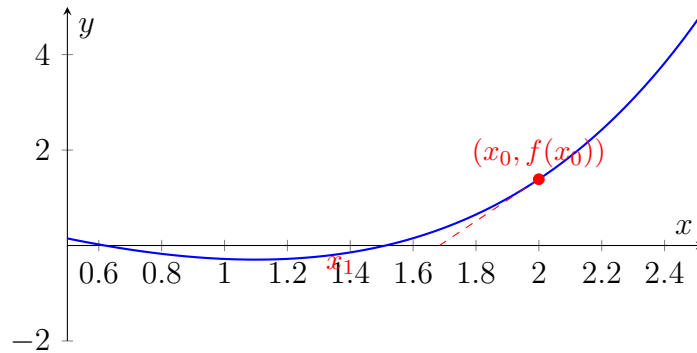
Proof. At each iteration, the interval is halved. After n iterations, $|I_n| = (b-a)/2^n$ and $|c_n - x^*| \leq |I_n|/2$. $\square$

2.3 Newton's method

Definition 2.3. Newton's iteration for $f(x) = 0$ is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Newton's method



Algorithm 2: Newton's method

Input: f, f', x_0 , tolerance ε , $N_{\max}$

Output: Approximation x_n

```

for  $n = 0, 1, \dots, N_{\max}$  do
     $x_{n+1} \leftarrow x_n - f(x_n)/f'(x_n);$ 
    if  $|x_{n+1} - x_n| < \varepsilon$  then
        return  $x_{n+1}$ 

```

Theorem 2.4 (Quadratic convergence of Newton's method). *If $f \in C^2$, $f(x^*) = 0$, $f'(x^*) \neq 0$ and x_0 is sufficiently close to x^* , then:*

$$|e_{n+1}| \leq \frac{M}{2m} |e_n|^2,$$

where $e_n = x_n - x^*$, $M = \sup |f''|$, $m = \inf |f'|$ in a neighbourhood of x^* .

Proof. By Taylor's theorem: $0 = f(x^*) = f(x_n) + f'(x_n)(x^* - x_n) + \frac{f''(\xi_n)}{2}(x^* - x_n)^2$. Dividing by $f'(x_n)$:

$$0 = \frac{f(x_n)}{f'(x_n)} + (x^* - x_n) + \frac{f''(\xi_n)}{2f'(x_n)} e_n^2.$$

Since $x_{n+1} = x_n - f(x_n)/f'(x_n)$, we get $e_{n+1} = x_{n+1} - x^* = -\frac{f''(\xi_n)}{2f'(x_n)} e_n^2$. $\square$

2.4 Secant method

Definition 2.5. We replace $f'(x_n)$ by a finite difference:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Theorem 2.6. *The order of convergence of the secant method is the **golden ratio** $\varphi = (1 + \sqrt{5})/2 \approx 1.618$.*

2.5 Fixed-point method

Definition 2.7. Find $x = g(x)$. Iteration: $x_{n+1} = g(x_n)$.

Theorem 2.8 (Banach Fixed-Point Theorem). *If $g : [a, b] \rightarrow [a, b]$ is a contraction ($|g'(x)| \leq L < 1$ on $[a, b]$), then g has a unique fixed point x^* and $x_n \rightarrow x^*$ with:*

$$|x_n - x^*| \leq \frac{L^n}{1 - L} |x_1 - x_0|.$$

2.6 Comparison of methods

Method	Order	Evaluations of f	Robustness
Bisection	1 (linear)	1/iter.	Very robust
Newton	2 (quadratic)	$f + f'$ /iter.	Sensitive to x_0
Secant	$\varphi \approx 1.618$	1/iter.	Moderate

2.7 Numerical example

Root of $f(x) = e^x - 3x$ with $x_0 = 2$ (Newton):

n	x_n	$f(x_n)$	$ x_n - x_{n-1} $
0	2.000000	1.389056	—
1	1.373418	0.225698	0.627
2	1.524247	-0.046	0.151
3	1.512127	-0.000374	0.012
4	1.512135	$< 10^{-9}$	8×10^{-6}

2.8 Implementation

Python

```

import numpy as np

def bisection(f, a, b, tol=1e-12):
    while b - a > tol:
        c = (a + b) / 2
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

def newton(f, df, x0, tol=1e-12, maxiter=100):
    x = x0
    for i in range(maxiter):
        dx = f(x) / df(x)
        x -= dx
        if abs(dx) < tol:
            return x, i+1
    return x, maxiter

def secant(f, x0, x1, tol=1e-12, maxiter=100):
    for i in range(maxiter):
        fx0, fx1 = f(x0), f(x1)
        x2 = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
        if abs(x2 - x1) < tol:
            return x2, i+1
        x0, x1 = x1, x2
    return x1, maxiter

# Test
f = lambda x: np.exp(x) - 3*x
df = lambda x: np.exp(x) - 3

print(f"Bisection : {bisection(f, 0, 1):.12f}")
x_newton, n = newton(f, df, 2.0)
print(f"Newton      : {x_newton:.12f} ({n} iterations)")
x_sec, n = secant(f, 0.0, 1.0)
print(f"Secante      : {x_sec:.12f} ({n} iterations)")

```

Julia

```

function newton(f, df, x0; tol=1e-12, maxiter=100)
    x = x0
    for i in 1:maxiter
        dx = f(x) / df(x)
        x -= dx
        abs(dx) < tol && return x, i
    end
end

```

```

    return x, maxiter
end

f(x) = exp(x) - 3x
df(x) = exp(x) - 3
x, n = newton(f, df, 2.0)
println("Newton: x = $x ($n iterations)")

```

2.9 Exercises

Exercise 2.1 (★). Find $\sqrt{2}$ by applying Newton's method to $f(x) = x^2 - 2$. How many iterations are needed for 15 decimal digits?

Exercise 2.2 (★★). Show that Newton's method applied to $f(x) = x^2$ converges linearly (double root). Generalise: what happens when $f'(x^*) = 0$?

Exercise 2.3 (★★). Prove that the order of the secant method is φ by setting $e_{n+1} \approx C e_n^p$ and solving $p^2 - p - 1 = 0$.

Exercise 2.4 (★★★ – Project). Implement Brent's method (combination of bisection + secant). Compare with the simpler methods on 10 test functions.

Key Formulas

$$\text{Bisection : } |e_n| \leq \frac{b-a}{2^{n+1}}$$

$$\text{Newton : } x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad |e_{n+1}| \leq C|e_n|^2$$

$$\text{Secant : } x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}, \quad \text{order } \varphi$$

Chapter 3

Direct Methods for Linear Systems

In 1810, Carl Friedrich Gauss publishes in *Disquisitio de elementis ellipticis Palladis* the elimination method that today bears his name: to solve a system of twelve equations in twelve unknowns from the orbit of the asteroid Pallas, he transforms the system by row operations until it reaches a triangular form solvable by back-substitution. Written cleanly, the same idea gives the LU decomposition. A century later, in 1910, André-Louis Cholesky — a French geodetic engineer killed in 1918 — discovers that for symmetric positive definite matrices, exploiting the symmetry halves the operation count. His method, published posthumously, remains the cornerstone of least-squares computation. This chapter develops the LU, $\text{LDL}^\top$, and Cholesky factorizations, analyzes their cost and numerical stability (pivots, perturbations), and presents the partial-pivoting strategies that guarantee robustness.

3.1 Motivating example

An electrical circuit with n nodes produces a system $Ax = b$ of n equations. For $n = 1000$, a systematic and efficient method is needed.

3.2 Gaussian elimination and LU factorisation

Definition 3.1 (LU factorisation). Find L lower triangular (with $\ell_{ii} = 1$) and U upper triangular such that $A = LU$. Then $Ax = b$ is solved by $Ly = b$ (forward substitution) followed by $Ux = y$ (back substitution).

Algorithm 3: LU factorisation (without pivoting)

Input: Matrix $A \in \mathbb{R}^{n \times n}$

Output: L, U such that $A = LU$

for $k = 1, \dots, n - 1$ **do**

for $i = k + 1, \dots, n$ **do**

$\ell_{ik} \leftarrow a_{ik}/a_{kk};$

for $j = k + 1, \dots, n$ **do**

$a_{ij} \leftarrow a_{ij} - \ell_{ik} \cdot a_{kj};$

$U \leftarrow$ upper triangular part of A ;

Theorem 3.2 (Existence of LU). *If all leading principal submatrices of A are invertible, then the LU factorisation exists and is unique.*

Theorem 3.3 (Cost). *The LU factorisation costs $\frac{2n^3}{3} + \mathcal{O}(n^2)$ floating-point operations.*

Proof. Step k performs $(n-k)^2$ multiplications and subtractions. Total: $\sum_{k=1}^{n-1} 2(n-k)^2 = 2 \sum_{j=1}^{n-1} j^2 = \frac{2n^3}{3} + \mathcal{O}(n^2)$. $\square$

3.3 Partial pivoting

Warning

Without pivoting, the algorithm can be unstable (division by a small pivot). **Partial pivoting** permutes rows to maximise $|a_{kk}|$.

One obtains $PA = LU$ where P is a permutation matrix.

Theorem 3.4 (Stability). *With partial pivoting, Gaussian elimination is **backward stable**:*

$$(A + \Delta A)\hat{x} = b, \quad \|\Delta A\| \leq g(n) \varepsilon_{\text{mach}} \|A\|,$$

where $g(n)$ is the growth factor (bounded by 2^{n-1} in theory, rarely exceeded in practice).

3.4 Cholesky factorisation

Theorem 3.5 (Cholesky factorisation). *If A is **symmetric positive definite** (SPD), then there exists a unique lower triangular matrix L with positive diagonal entries such that $A = LL^\top$.*

Proof. By induction on n . Write $A = \begin{pmatrix} a_{11} & \mathbf{v}^\top \\ \mathbf{v} & A' \end{pmatrix}$. Since A is SPD, $a_{11} > 0$. Set $\ell_{11} = \sqrt{a_{11}}$, $\mathbf{l} = \mathbf{v}/\ell_{11}$. Then $A' - \mathbf{l}\mathbf{l}^\top$ is SPD of size $(n-1) \times (n-1)$ and we apply the induction hypothesis. $\square$

Algorithm 4: Cholesky factorisation

Input: A SPD, $n \times n$

Output: L such that $A = LL^\top$

for $j = 1, \dots, n$ **do**

$\ell_{jj} \leftarrow \sqrt{a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2};$

for $i = j + 1, \dots, n$ **do**

$\ell_{ij} \leftarrow \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} \right);$

Theorem 3.6 (Cost of Cholesky). $\frac{n^3}{3} + \mathcal{O}(n^2)$: *half the cost of LU.*

3.5 Numerical example

$$A = \begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix}, b = \begin{pmatrix} 8 \\ 9 \end{pmatrix}. \text{ Cholesky: } L = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}.$$

Forward substitution $Ly = b$: $y_1 = 4$, $y_2 = (9 - 1 \cdot 4)/2 = 2.5$. Back substitution $L^\top x = y$: $x_2 = 2.5/2 = 1.25$, $x_1 = (4 - 1 \cdot 1.25)/2 = 1.375$.

3.6 Implementation

Python

```
import numpy as np
from scipy.linalg import lu, cholesky, solve_triangular

# LU
A = np.array([[2., 1, 1], [4, 3, 3], [8, 7, 9]])
b = np.array([4., 10, 24])
P, L, U = lu(A)
y = solve_triangular(L, P @ b, lower=True)
x = solve_triangular(U, y)
print(f"LU: x = {x}")

# Cholesky
A_spd = np.array([[4., 2], [2, 5]])
b_spd = np.array([8., 9])
L_chol = cholesky(A_spd, lower=True)
y = solve_triangular(L_chol, b_spd, lower=True)
x = solve_triangular(L_chol.T, y)
print(f"Cholesky: x = {x}")
print(f"Cout LU (n=100): ~{2*100**3//3:.0f} flops")
```

Julia

```
using LinearAlgebra
A = [2.0 1 1; 4 3 3; 8 7 9]
b = [4.0, 10, 24]
F = lu(A)
x = F \ b
println("LU: x = $x")

A_spd = [4.0 2; 2 5]
b_spd = [8.0, 9]
C = cholesky(A_spd)
x = C \ b_spd
println("Cholesky: x = $x")
```

3.7 Exercises

Exercise 3.1 (*). Factorise $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 8 & 14 \\ 3 & 14 & 34 \end{pmatrix}$ into LU by hand. Verify.

Exercise 3.2 (**). Show that if A is SPD, all pivots in Gaussian elimination are positive (no pivoting needed).

Exercise 3.3 (**). Compare the computation times of LU vs. Cholesky for $n = 100, 500, 1000, 2000$ on random SPD matrices. Verify the ratio ~ 2 .

Exercise 3.4 (*** – Project). Implement the banded LU factorisation for tridiagonal matrices (Thomas algorithm). Apply to the discretisation of $-u'' = f$.

Key Formulas

$$A = LU \quad (\text{cost } \frac{2n^3}{3})$$

$$A = LL^\top \quad (\text{Cholesky, cost } \frac{n^3}{3})$$

$$PA = LU \quad (\text{partial pivoting})$$

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|, \quad \frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}$$

Chapter 4

Iterative Methods for Linear Systems

When utility engineers at EDF must solve a linear system to simulate a national power grid — several million unknowns — direct methods become infeasible: LU costs $O(n^3)$ and fills the memory. The escape, anticipated by Jacobi in 1845 and developed by Gauss and Seidel, is to abandon the idea of finding the exact solution in finitely many steps and accept a sequence of approximations that converges to it. For a sparse matrix, this trade is favorable: each iteration costs only $O(\text{nnz})$, and a few dozen iterations typically suffice. This chapter introduces the classical iterative methods (Jacobi, Gauss-Seidel, successive over-relaxation), then the modern Krylov methods (conjugate gradient, GMRES) that dominate large-scale scientific computing. Convergence theory rests on the spectral radius of the iteration matrix.

4.1 Motivating example

In finite element methods, the matrix A is sparse ($\mathcal{O}(n)$ nonzero entries out of n^2). LU costs $\mathcal{O}(n^3)$, which is unacceptable for $n = 10^6$. Iterative methods exploit the sparse structure: each iteration costs $\mathcal{O}(n)$.

4.2 General principle

Definition 4.1 (Splitting method). We write $A = M - N$ with M easily invertible. The iteration is:

$$Mx^{(k+1)} = Nx^{(k)} + b, \quad \text{i.e.} \quad x^{(k+1)} = Gx^{(k)} + c,$$

where $G = M^{-1}N$ is the **iteration matrix** and $c = M^{-1}b$.

Theorem 4.2 (Convergence). *The method converges for every $x^{(0)}$ if and only if $\rho(G) < 1$ (spectral radius).*

Proof. $e^{(k)} = x^{(k)} - x^* = G^k e^{(0)}$. Now $\|G^k\| \rightarrow 0 \iff \rho(G) < 1$. □

4.3 Jacobi method

Definition 4.3. $A = D - E - F$ (diagonal, lower triangular, upper triangular). $M = D$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Iteration matrix: $G_J = D^{-1}(E + F)$.

4.4 Gauss–Seidel method

Definition 4.4. $M = D - E$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

Iteration matrix: $G_{GS} = (D - E)^{-1}F$.

Theorem 4.5 (Convergence for SPD matrices). *If A is SPD, Gauss–Seidel converges.*

Theorem 4.6 (Convergence for diagonally dominant matrices). *If A is strictly diagonally dominant ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$), then both Jacobi and Gauss–Seidel converge.*

4.5 Successive over-relaxation (SOR)

Definition 4.7. We introduce a parameter $\omega \in (0, 2)$:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

$\omega = 1$: Gauss–Seidel. $\omega > 1$: over-relaxation. $\omega < 1$: under-relaxation.

Theorem 4.8. *SOR converges if and only if $0 < \omega < 2$. The optimal ω depends on $\rho(G_J)$.*

4.6 Conjugate gradient method

Definition 4.9. For A SPD, solving $Ax = b$ is equivalent to minimising $\phi(x) = \frac{1}{2}x^\top Ax - b^\top x$.

Algorithm 5: Conjugate gradient

Input: A SPD, b , x_0 , tolerance ε

Output: $x \approx A^{-1}b$

$r_0 \leftarrow b - Ax_0$, $p_0 \leftarrow r_0$;

for $k = 0, 1, 2, \dots$ **do**

$\alpha_k \leftarrow \frac{r_k^\top r_k}{p_k^\top A p_k}$;

$x_{k+1} \leftarrow x_k + \alpha_k p_k$;

$r_{k+1} \leftarrow r_k - \alpha_k A p_k$;

if $\|r_{k+1}\| < \varepsilon$ **then**

return x_{k+1}

$\beta_k \leftarrow \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}$;

$p_{k+1} \leftarrow r_{k+1} + \beta_k p_k$;

Theorem 4.10 (Convergence of the conjugate gradient method). *In exact arithmetic, CG converges in at most n iterations. The error satisfies:*

$$\|e_k\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A, \quad \kappa = \text{cond}_2(A).$$

4.7 Implementation

Python

```
import numpy as np

def jacobi(A, b, x0, tol=1e-10, maxiter=1000):
    D = np.diag(np.diag(A))
    R = A - D
    x = x0.copy()
    for k in range(maxiter):
        x_new = np.linalg.solve(D, b - R @ x)
        if np.linalg.norm(x_new - x) < tol:
            return x_new, k+1
        x = x_new
    return x, maxiter

def conjugate_gradient(A, b, x0, tol=1e-10, maxiter=1000):
    x = x0.copy()
    r = b - A @ x
    p = r.copy()
    rs_old = r @ r
    for k in range(maxiter):
        Ap = A @ p
        alpha = rs_old / (p @ Ap)
        x += alpha * p
        r -= alpha * Ap
        rs_new = r @ r
```

```

    if np.sqrt(rs_new) < tol:
        return x, k+1
    p = r + (rs_new / rs_old) * p
    rs_old = rs_new
    return x, maxiter

# Test : matrice tridiagonale
n = 100
A = np.diag(2*np.ones(n)) - np.diag(np.ones(n-1), 1) -
    ↪ np.diag(np.ones(n-1), -1)
b = np.ones(n)
x0 = np.zeros(n)

x_j, nj = jacobi(A, b, x0)
x_cg, ncg = conjugate_gradient(A, b, x0)
print(f"Jacobi: {nj} iterations")
print(f"CG:      {ncg} iterations")

```

Julia

```

function cg(A, b, x0; tol=1e-10, maxiter=1000)
    x = copy(x0)
    r = b - A * x
    p = copy(r)
    rs = dot(r, r)
    for k in 1:maxiter
        Ap = A * p
        alpha = rs / dot(p, Ap)
        x .+= alpha .* p
        r .-= alpha .* Ap
        rs_new = dot(r, r)
        sqrt(rs_new) < tol && return x, k
        p .= r .+ (rs_new / rs) .* p
        rs = rs_new
    end
    return x, maxiter
end

```

4.8 Exercises

Exercise 4.1 (*). Apply 3 iterations of Jacobi and Gauss–Seidel to the system $\begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $x^{(0)} = (0, 0)^\top$.

Exercise 4.2 (**). Show that for a tridiagonal matrix T_n of size n , $\rho(G_J) = \cos(\pi/(n+1))$. Deduce the number of Jacobi iterations needed for a precision ε .

Exercise 4.3 (*** – Project). Compare Jacobi, Gauss–Seidel, SOR and CG for the 2D discretisation of the Laplacian $-\Delta u = f$ on $[0, 1]^2$. Plot the number of iterations as a function of n .

Key Formulas

$$\text{Jacobi : } x^{(k+1)} = D^{-1}(b - (A - D)x^{(k)})$$

$$\text{Gauss–Seidel : } G = (D - E)^{-1}F$$

$$\text{CG : } \|e_k\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A$$

Chapter 5

Polynomial Interpolation

In 1901, the German mathematician Carl Runge publishes a surprising result: if the function $f(x) = 1/(1 + 25x^2)$ is interpolated on $[-1, 1]$ by a polynomial of degree n at equally spaced nodes, the interpolation error *diverges* as $n \rightarrow \infty$. The more sample points one takes, the worse the approximation near the endpoints. The Runge phenomenon, as it came to be called, stopped naive polynomial interpolation in physics and astronomy and steered mathematicians toward Chebyshev nodes, splines, and the modern theory of numerical stability. This chapter builds polynomial interpolation (Lagrange form, Newton form, divided differences), analyzes the error, presents cubic splines — the workhorse of CAD smoothing — and ends with the basis-function approach used in finite-element interpolation.

5.1 Motivating example

We have temperature measurements at 5 time instants. We want to estimate the temperature at an intermediate time. Polynomial interpolation constructs a polynomial passing exactly through the measured points.

5.2 The interpolation problem

Theorem 5.1 (Existence and uniqueness). *Given $n+1$ distinct points $(x_0, y_0), \dots, (x_n, y_n)$, there exists a **unique** polynomial $p_n \in \mathcal{P}_n$ such that $p_n(x_i) = y_i$ for all i .*

Proof. The Vandermonde matrix $V_{ij} = x_i^j$ is invertible since $\det(V) = \prod_{i>j} (x_i - x_j) \neq 0$. $\square$

5.3 Lagrange form

Definition 5.2.

$$p_n(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

The L_i are the **Lagrange basis polynomials**: $L_i(x_j) = \delta_{ij}$.

5.4 Newton form

Definition 5.3 (Divided differences).

$$f[x_i] = f(x_i), \quad f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Definition 5.4 (Newton form).

$$p_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + \dots + f[x_0, \dots, x_n] \prod_{j=0}^{n-1} (x - x_j).$$

Evaluation by the generalised Horner algorithm: $\mathcal{O}(n)$ operations.

5.5 Interpolation error

Theorem 5.5 (Interpolation error). *If $f \in C^{n+1}([a, b])$, then for every $x \in [a, b]$:*

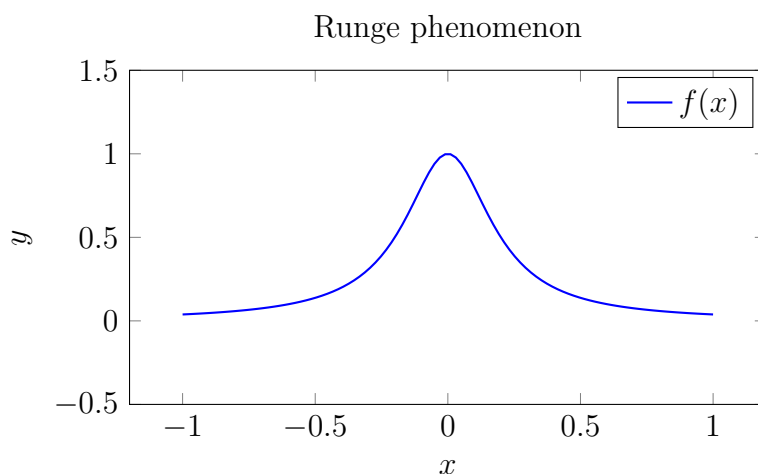
$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

where $\xi_x \in (a, b)$ depends on x .

Proof. Set $w(t) = \prod (t - x_i)$ and $\varphi(t) = f(t) - p_n(t) - \lambda w(t)$ with λ chosen so that $\varphi(x) = 0$ (for a fixed $x \neq x_i$). Then φ vanishes at $n+2$ points. By Rolle's theorem applied $n+1$ times, $\varphi^{(n+1)}$ vanishes at some point ξ . Since $\varphi^{(n+1)} = f^{(n+1)} - \lambda(n+1)!$, we get $\lambda = f^{(n+1)}(\xi)/(n+1)!$. $\square$

Warning

The **Runge phenomenon**: for $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ with equidistant nodes, the interpolation error *diverges* as $n \rightarrow \infty$ near the endpoints.



5.6 Chebyshev nodes

Definition 5.6. The Chebyshev nodes on $[-1, 1]$:

$$x_k = \cos\left(\frac{2k+1}{2(n+1)}\pi\right), \quad k = 0, \dots, n.$$

They minimise $\max_{x \in [-1, 1]} |\prod (x - x_i)|$.

Theorem 5.7. *With Chebyshev nodes: $\max |w(x)| \leq 2^{-n}$, which avoids the Runge phenomenon for analytic functions.*

5.7 Cubic splines

Definition 5.8 (Cubic interpolating spline). $s \in C^2([a, b])$, $s|_{[x_i, x_{i+1}]} \in \mathcal{P}_3$, $s(x_i) = y_i$. Boundary conditions:

- **Natural:** $s''(x_0) = s''(x_n) = 0$.
- **Clamped:** $s'(x_0) = f'(x_0)$, $s'(x_n) = f'(x_n)$.

Theorem 5.9 (Cubic spline error). *If $f \in C^4$ and $h = \max(x_{i+1} - x_i)$:*

$$\|f - s\|_\infty \leq \frac{5}{384} h^4 \|f^{(4)}\|_\infty.$$

5.8 Implementation

Python

```
import numpy as np
from scipy.interpolate import lagrange, CubicSpline
import matplotlib.pyplot as plt

# Interpolation de Lagrange
x_nodes = np.array([-1, -0.5, 0, 0.5, 1])
f = lambda x: 1 / (1 + 25*x**2)
y_nodes = f(x_nodes)
p = lagrange(x_nodes, y_nodes)

x_fine = np.linspace(-1, 1, 300)
plt.figure(figsize=(10, 5))
plt.plot(x_fine, f(x_fine), 'b-', label='f(x)', lw=2)
plt.plot(x_fine, p(x_fine), 'r--', label=f'Lagrange deg
↪ {len(x_nodes)-1}')
plt.plot(x_nodes, y_nodes, 'ko', markersize=8)
plt.legend(); plt.title("Interpolation de Lagrange")
plt.savefig("ch05_lagrange.pdf"); plt.show()
```

```
# Noeuds de Tchebychev
n = 15
cheb = np.cos((2*np.arange(n+1)+1)/(2*(n+1))*np.pi)
y_cheb = f(cheb)
p_cheb = lagrange(cheb, y_cheb)
print(f"Erreur equidist (n=15):
↳ {np.max(np.abs(f(x_fine)-lagrange(np.linspace(-1,1,16),
↳ f(np.linspace(-1,1,16)))(x_fine)):.4f}")
print(f"Erreur Tchebychev (n=15):
↳ {np.max(np.abs(f(x_fine)-p_cheb(x_fine)):.6f}")

# Spline cubique
cs = CubicSpline(x_nodes, y_nodes)
print(f"Spline en 0.25: {cs(0.25):.6f}, exact: {f(0.25):.6f}")
```

Julia

```
using Interpolations

f(x) = 1 / (1 + 25x^2)
nodes = [-1.0, -0.5, 0, 0.5, 1.0]
vals = f.(nodes)

# Differences divisees (Newton)
function divided_diff(x, y)
    n = length(x)
    F = copy(y)
    for j in 2:n, i in n:-1:j
        F[i] = (F[i] - F[i-1]) / (x[i] - x[i-j+1])
    end
    return F
end

println("Coefs Newton: ", divided_diff(nodes, vals))
```

5.9 Exercises

Exercise 5.1 (★). Construct the Lagrange polynomial passing through $(0, 1)$, $(1, 3)$, $(2, 7)$ and verify.

Exercise 5.2 (★★). Prove Theorem 5.5 in detail.

Exercise 5.3 (★★). Compare the interpolation error with equidistant nodes vs. Chebyshev nodes for $f(x) = 1/(1 + 25x^2)$, $n = 5, 10, 15, 20$.

Exercise 5.4 (★★★ – Project). Implement Hermite interpolation (data for both f and f'). Apply to the construction of Bézier curves.

Key Formulas

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$
$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod (x - x_i)$$
$$x_k^{\text{Cheb}} = \cos\left(\frac{2k+1}{2(n+1)}\pi\right)$$
$$\|f - s\|_\infty \leq \frac{5}{384} h^4 \|f^{(4)}\|_\infty$$

Chapter 6

Approximation — Least Squares and Chebyshev

In 1854, Pafnouti Chebyshev visits the textile mills of Lille and observes that the metal parts of the looms wear unevenly because the linkages produce only approximately straight-line motion. The underlying theoretical problem: among all monic polynomials of a given degree, find the one whose maximum absolute value on an interval is minimal. Chebyshev solves the problem, and the resulting family — the Chebyshev polynomials — becomes one of the most fertile discoveries of nineteenth-century applied analysis. The same mathematics now optimizes digital filters, spectral methods for PDEs, and rational approximation in scientific computing. This chapter covers least-squares approximation and minimax approximation, presents the classical orthogonal polynomials (Legendre, Chebyshev, Hermite), and applies them to quadrature and PDE discretization.

6.1 Introduction

In many practical situations, we have a set of data points and wish to find a function that *best represents* them. Unlike interpolation, which requires the approximating function to pass exactly through every data point, approximation seeks the function that is *closest* in some well-defined sense.

Definition 6.1 (Approximation problem). Given data points (x_i, y_i) , $i = 0, 1, \dots, m$, and a family of functions $\{g(x; a_0, \dots, a_n)\}$ with $n < m$, the **approximation problem** consists of finding the parameters $a_0, \dots, a_n$ that minimize a measure of the discrepancy between $g(x_i; a_0, \dots, a_n)$ and y_i .

The two principal approaches are:

- **Least squares approximation:** minimize the sum of squared residuals;
- **Chebyshev (minimax) approximation:** minimize the maximum absolute error.

6.2 Discrete Least Squares

6.2.1 Problem formulation

Definition 6.2 (Discrete least squares problem). Given $m + 1$ data points (x_i, y_i) for $i = 0, \dots, m$ and a basis of functions $\varphi_0, \varphi_1, \dots, \varphi_n$ with $n < m$, we seek coefficients $a_0, a_1, \dots, a_n$ minimizing

$$S(a_0, \dots, a_n) = \sum_{i=0}^m \left(y_i - \sum_{j=0}^n a_j \varphi_j(x_i) \right)^2.$$

Remark 6.3. The condition $n < m$ is essential: the system is *overdetermined*, so we cannot in general satisfy all equations exactly. When $n = m$, we recover the interpolation problem.

6.2.2 Normal equations

Theorem 6.4 (Normal equations). *The least squares solution satisfies the **normal equations**:*

$$\mathbf{A}^\top \mathbf{A} \mathbf{a} = \mathbf{A}^\top \mathbf{y},$$

where $\mathbf{A}$ is the $(m + 1) \times (n + 1)$ matrix with entries $A_{ij} = \varphi_j(x_i)$, $\mathbf{a} = (a_0, \dots, a_n)^\top$, and $\mathbf{y} = (y_0, \dots, y_m)^\top$.

Proof. Define $\mathbf{r} = \mathbf{y} - \mathbf{A}\mathbf{a}$ (the residual vector). Then

$$S(\mathbf{a}) = \|\mathbf{r}\|_2^2 = (\mathbf{y} - \mathbf{A}\mathbf{a})^\top (\mathbf{y} - \mathbf{A}\mathbf{a}) = \mathbf{y}^\top \mathbf{y} - 2\mathbf{a}^\top \mathbf{A}^\top \mathbf{y} + \mathbf{a}^\top \mathbf{A}^\top \mathbf{A} \mathbf{a}.$$

Setting the gradient with respect to $\mathbf{a}$ to zero:

$$\nabla_{\mathbf{a}} S = -2\mathbf{A}^\top \mathbf{y} + 2\mathbf{A}^\top \mathbf{A} \mathbf{a} = \mathbf{0}.$$

This gives $\mathbf{A}^\top \mathbf{A} \mathbf{a} = \mathbf{A}^\top \mathbf{y}$.

Since S is a convex quadratic form (the Hessian $2\mathbf{A}^\top \mathbf{A}$ is positive semi-definite), any critical point is a global minimum. If the columns of $\mathbf{A}$ are linearly independent, then $\mathbf{A}^\top \mathbf{A}$ is positive definite and the solution is unique. $\square$

Example 6.5 (Linear least squares fit). Given the data

x_i	0	1	2	3	4
y_i	1.0	2.1	2.9	4.2	4.8

we fit $g(x) = a_0 + a_1x$. The matrix $\mathbf{A}$ and the normal equations are:

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}, \quad \mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 5 & 10 \\ 10 & 30 \end{pmatrix}, \quad \mathbf{A}^\top \mathbf{y} = \begin{pmatrix} 15.0 \\ 38.8 \end{pmatrix}.$$

Solving: $a_0 = 1.08$, $a_1 = 0.96$, so $g(x) = 1.08 + 0.96x$.

Warning

The matrix $\mathbf{A}^\top \mathbf{A}$ can be very ill-conditioned, especially when using monomials $\varphi_j(x) = x^j$ of high degree. This is why orthogonal polynomials or QR factorization are preferred in practice.

6.2.3 Polynomial least squares via QR factorization

Proposition 6.6. *If $\mathbf{A} = \mathbf{QR}$ is the (thin) QR factorization of $\mathbf{A}$ where $\mathbf{Q} \in \mathbb{R}^{(m+1) \times (n+1)}$ has orthonormal columns and $\mathbf{R} \in \mathbb{R}^{(n+1) \times (n+1)}$ is upper triangular, then the least squares solution is*

$$\mathbf{R} \mathbf{a} = \mathbf{Q}^\top \mathbf{y}.$$

Proof. From $\mathbf{A} = \mathbf{QR}$, we get $\mathbf{A}^\top \mathbf{A} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{QR} = \mathbf{R}^\top \mathbf{R}$ and $\mathbf{A}^\top \mathbf{y} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{y}$. The normal equations become $\mathbf{R}^\top \mathbf{R} \mathbf{a} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{y}$. Since $\mathbf{R}$ is invertible, we may multiply on the left by $(\mathbf{R}^\top)^{-1}$ to obtain $\mathbf{R} \mathbf{a} = \mathbf{Q}^\top \mathbf{y}$. $\square$

6.3 Continuous Least Squares and Orthogonal Polynomials

6.3.1 Continuous least squares

Definition 6.7 (Continuous least squares). Given a weight function $w(x) > 0$ on $[a, b]$ and a function $f \in C([a, b])$, we seek a polynomial $p_n \in \mathcal{P}_n$ minimizing

$$\|f - p_n\|_w^2 = \int_a^b w(x) [f(x) - p_n(x)]^2 dx.$$

The solution satisfies the continuous normal equations:

$$\sum_{j=0}^n a_j \underbrace{\int_a^b w(x) \varphi_j(x) \varphi_k(x) dx}_{\langle \varphi_j, \varphi_k \rangle_w} = \underbrace{\int_a^b w(x) f(x) \varphi_k(x) dx}_{\langle f, \varphi_k \rangle_w}, \quad k = 0, \dots, n.$$

6.3.2 Orthogonal polynomials

Definition 6.8 (Orthogonal polynomials). A sequence $\{p_k\}_{k \geq 0}$ of polynomials (with $\deg p_k = k$) is **orthogonal** with respect to the inner product $\langle \cdot, \cdot \rangle_w$ if

$$\langle p_j, p_k \rangle_w = \int_a^b w(x) p_j(x) p_k(x) dx = 0 \quad \text{for } j \neq k.$$

Theorem 6.9 (Three-term recurrence). *Orthogonal polynomials satisfy a recurrence of the form*

$$p_{k+1}(x) = (\alpha_k x + \beta_k) p_k(x) - \gamma_k p_{k-1}(x), \quad k \geq 1,$$

with $p_0(x) = 1$ and $p_1(x) = \alpha_0 x + \beta_0$.

Example 6.10 (Classical orthogonal polynomials).

Name	Interval	$w(x)$	Notation
Legendre	$[-1, 1]$	1	$P_k(x)$
Chebyshev	$[-1, 1]$	$(1 - x^2)^{-1/2}$	$T_k(x)$
Laguerre	$[0, \infty)$	e^{-x}	$L_k(x)$
Hermite	$(-\infty, \infty)$	e^{-x^2}	$H_k(x)$

Proposition 6.11. *If $\{p_0, p_1, \dots, p_n\}$ is an orthogonal basis, the best approximation is*

$$f_n(x) = \sum_{k=0}^n c_k p_k(x), \quad c_k = \frac{\langle f, p_k \rangle_w}{\langle p_k, p_k \rangle_w}.$$

The normal equations decouple because the Gram matrix is diagonal.

6.4 Chebyshev Polynomials

Definition 6.12 (Chebyshev polynomials). The Chebyshev polynomials of the first kind are defined by

$$T_k(x) = \cos(k \arccos x), \quad x \in [-1, 1], \quad k = 0, 1, 2, \dots$$

They satisfy the recurrence $T_0(x) = 1$, $T_1(x) = x$, $T_{k+1}(x) = 2x T_k(x) - T_{k-1}(x)$.

Theorem 6.13 (Properties of Chebyshev polynomials). 1. T_k has exactly k zeros on $(-1, 1)$: $x_j = \cos\left(\frac{(2j-1)\pi}{2k}\right)$, $j = 1, \dots, k$.

2. $|T_k(x)| \leq 1$ for all $x \in [-1, 1]$.

3. T_k attains the values ± 1 at $k+1$ extremal points: $x_j^* = \cos\left(\frac{j\pi}{k}\right)$, $j = 0, \dots, k$.

4. Orthogonality:
$$\int_{-1}^1 \frac{T_j(x) T_k(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0 & j \neq k, \\ \pi & j = k = 0, \\ \pi/2 & j = k \neq 0. \end{cases}$$

Theorem 6.14 (Minimax property). Among all monic polynomials of degree n , the polynomial $\tilde{T}_n(x) = 2^{1-n} T_n(x)$ has the smallest uniform norm on $[-1, 1]$:

$$\max_{x \in [-1, 1]} |\tilde{T}_n(x)| = 2^{1-n}.$$

6.5 Best Uniform Approximation (Chebyshev / Minimax)

Definition 6.15 (Best uniform approximation). Given $f \in C([a, b])$, the **best uniform (minimax) approximation** of degree n is the polynomial p_n^* satisfying

$$\|f - p_n^*\|_\infty = \min_{p \in \mathcal{P}_n} \|f - p\|_\infty = \min_{p \in \mathcal{P}_n} \max_{x \in [a, b]} |f(x) - p(x)|.$$

Theorem 6.16 (Chebyshev equioscillation theorem). *A polynomial $p_n^* \in \mathcal{P}_n$ is the best uniform approximation to $f \in C([a, b])$ if and only if the error $e(x) = f(x) - p_n^*(x)$ **equioscillates** at least $n + 2$ times: there exist $n + 2$ points*

$$a \leq x_0 < x_1 < \cdots < x_{n+1} \leq b$$

such that

$$e(x_j) = (-1)^j \sigma \|e\|_\infty, \quad j = 0, 1, \dots, n+1,$$

where $\sigma = \pm 1$.

Remark 6.17. The equioscillation theorem guarantees both existence and uniqueness of the best uniform approximation. The Remez algorithm is a classical iterative method for computing it.

6.6 Near-best Approximation via Chebyshev Interpolation

Theorem 6.18 (Near-optimality of Chebyshev interpolation). *Let p_n^{Cheb} be the interpolation polynomial at Chebyshev nodes and p_n^* the best uniform approximation. Then*

$$\|f - p_n^{\text{Cheb}}\|_\infty \leq (1 + \Lambda_n^{\text{Cheb}}) \|f - p_n^*\|_\infty,$$

where $\Lambda_n^{\text{Cheb}} = \mathcal{O}(\ln n)$ is the Lebesgue constant for Chebyshev nodes.

Method

Practical guideline: For most applications, Chebyshev interpolation provides a near-optimal approximation that is much simpler to compute than the true minimax approximation.

6.7 Python Implementation

Python — Least squares via normal equations and polyfit

```
import numpy as np
import matplotlib.pyplot as plt

# Data points
x = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([1.0, 2.1, 2.9, 4.2, 4.8])

# --- Method 1: Normal equations ---
A = np.column_stack([np.ones_like(x), x])
ATA = A.T @ A
ATy = A.T @ y
a_normal = np.linalg.solve(ATA, ATy)
```

```

print("Normal equations: a0 = {:.4f}, a1 = {:.4f}".format(*a_normal))

# --- Method 2: QR factorization ---
Q, R = np.linalg.qr(A)
a_qr = np.linalg.solve(R, Q.T @ y)
print("QR method:          a0 = {:.4f}, a1 = {:.4f}".format(*a_qr))

# --- Method 3: numpy.polyfit ---
coeffs = np.polyfit(x, y, 1) # returns [a1, a0]
print("polyfit:          a0 = {:.4f}, a1 = {:.4f}".format(coeffs[1],
    ↪ coeffs[0]))

# --- Higher degree fit ---
for deg in [1, 2, 3]:
    c = np.polyfit(x, y, deg)
    p = np.poly1d(c)
    residual = np.sqrt(np.sum((y - p(x))**2))
    print(f"Degree {deg}: residual = {residual:.6f}")

# --- Plot ---
xfine = np.linspace(-0.5, 4.5, 200)
plt.figure(figsize=(8, 5))
plt.plot(x, y, 'ko', markersize=8, label='Data')
for deg, ls in [(1, '-'), (2, '--'), (3, ':')]:
    c = np.polyfit(x, y, deg)
    plt.plot(xfine, np.polyval(c, xfine), ls,
        label=f'Degree {deg}')
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.legend()
plt.title('Least squares polynomial fits')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('least_squares_fit.pdf')
plt.show()

```

Python — Chebyshev approximation

```

import numpy as np
from numpy.polynomial import chebyshev as C

# Approximate  $f(x) = \exp(x)$  on  $[-1, 1]$ 
f = lambda x: np.exp(x)
n = 10

# Chebyshev interpolation nodes
k = np.arange(n + 1)
nodes = np.cos((2*k + 1) * np.pi / (2*(n + 1)))
values = f(nodes)

```

```
# Chebyshev coefficients
coeffs = C.chebfit(nodes, values, n)

# Evaluate and compare
xfine = np.linspace(-1, 1, 1000)
approx = C.chebval(xfine, coeffs)
error = np.max(np.abs(f(xfine) - approx))
print(f"Max error (degree {n}): {error:.2e}")
```

6.8 Exercises

Exercise 6.1 (★). Given the data points $(0, 1)$, $(1, 3)$, $(2, 4)$, $(3, 5)$, find the least squares line $y = a_0 + a_1x$.

1. Write the matrix $\mathbf{A}$ and the normal equations $\mathbf{A}^\top \mathbf{A} \mathbf{a} = \mathbf{A}^\top \mathbf{y}$.
2. Solve the system.
3. Compute the residual $\|\mathbf{y} - \mathbf{A}\mathbf{a}\|_2$.

Exercise 6.2 (★). Fit a quadratic $y = a_0 + a_1x + a_2x^2$ to the data

x_i	-2	-1	0	1	2	3
y_i	5.1	2.2	0.9	1.8	5.3	10.1

using the normal equations. Compare with `numpy.polyfit`.

Exercise 6.3 (★★). Consider the Vandermonde matrix $A_{ij} = x_i^j$ for $m + 1$ equispaced points on $[0, 1]$.

1. For $m = 10$ and $n = 2, 4, 6, 8$, compute $\text{cond}_2(\mathbf{A}^\top \mathbf{A})$.
2. Compare with $\text{cond}_2(\mathbf{R})$ from the QR factorization of $\mathbf{A}$.
3. What do you conclude about the numerical stability of normal equations?

Exercise 6.4 (★★). 1. Verify that $T_4(x) = 8x^4 - 8x^2 + 1$ using the recurrence relation.

2. Compute its four zeros.
3. Show that $\|T_4\|_\infty = 1$ on $[-1, 1]$ and identify the five extremal points.

Exercise 6.5 (★★★). Implement a simplified Remez algorithm:

1. Choose $n + 2$ initial reference points (e.g., Chebyshev extrema).
2. Solve the equioscillation system for p_n and the leveled error E .
3. Find the point of maximum error and exchange it with the worst reference point.

4. Repeat until convergence.

Test on $f(x) = |x|$ on $[-1, 1]$ with $n = 2, 4, 6$.

Exercise 6.6 (***). Consider the weighted least squares problem: minimize

$$S_w(\mathbf{a}) = \sum_{i=0}^m w_i \left(y_i - \sum_{j=0}^n a_j \varphi_j(x_i) \right)^2$$

where $w_i > 0$ are given weights.

1. Derive the weighted normal equations.
2. Show that they can be written as $\mathbf{A}^\top \mathbf{W} \mathbf{A} \mathbf{a} = \mathbf{A}^\top \mathbf{W} \mathbf{y}$ where $\mathbf{W} = \text{diag}(w_0, \dots, w_m)$.
3. Implement in Python and test with data where some points should be trusted more.

Summary of Key Formulas

Key Formulas

Normal equations:

$$\mathbf{A}^\top \mathbf{A} \mathbf{a} = \mathbf{A}^\top \mathbf{y}$$

QR approach:

$$\mathbf{R} \mathbf{a} = \mathbf{Q}^\top \mathbf{y} \quad (\mathbf{A} = \mathbf{Q} \mathbf{R})$$

Continuous least squares with orthogonal polynomials:

$$c_k = \frac{\langle f, p_k \rangle_w}{\langle p_k, p_k \rangle_w}, \quad f_n(x) = \sum_{k=0}^n c_k p_k(x)$$

Chebyshev polynomials:

$$T_k(x) = \cos(k \arccos x), \quad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x)$$

Equioscillation theorem: p_n^* is the best uniform approximation $\iff$ the error equioscillates at $\geq n + 2$ points.

Near-best bound:

$$\|f - p_n^{\text{Cheb}}\|_\infty \leq (1 + \Lambda_n^{\text{Cheb}}) \|f - p_n^*\|_\infty, \quad \Lambda_n^{\text{Cheb}} = \mathcal{O}(\ln n)$$

Chapter 7

Numerical Differentiation and Integration

When UK Met Office engineers begin digital weather forecasting in the 1950s, they quickly discover that trapezoidal quadrature is too expensive at the precision required. Werner Romberg, a Swedish mathematician, proposes in 1955 an elegant fix: combine trapezoidal estimates at multiple step sizes so as to cancel successive orders of error — Richardson extrapolation applied to quadrature. The result is that with a logarithmic number of function evaluations, one achieves the precision that would have required thousands at brute force. This chapter covers numerical differentiation (finite differences, higher-order derivatives, stability), Newton-Cotes quadrature (trapezoidal, Simpson), Romberg extrapolation, and adaptive quadrature — the default algorithm in `scipy.integrate`.

7.1 Introduction

When an analytical expression for a derivative or integral is unavailable—or the function is known only through data points—we resort to *numerical* methods. This chapter covers finite difference formulas for derivatives and the classical quadrature rules for definite integrals.

7.2 Numerical Differentiation

7.2.1 Finite difference formulas

Definition 7.1 (Finite differences). Given a function f and a step size $h > 0$:

- **Forward difference:** $f'(x) \approx \frac{f(x+h) - f(x)}{h}$
- **Backward difference:** $f'(x) \approx \frac{f(x) - f(x-h)}{h}$
- **Centered difference:** $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$

Theorem 7.2 (Truncation errors for finite differences). *If f is sufficiently smooth:*

1. **Forward/backward difference:**

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(\xi_1), \quad \xi_1 \in (x, x+h).$$

The error is $\mathcal{O}(h)$.

2. **Centered difference:**

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(\xi_2), \quad \xi_2 \in (x-h, x+h).$$

The error is $\mathcal{O}(h^2)$.

3. **Second derivative (centered):**

$$\frac{f(x+h) - 2f(x) + f(x-h)}{h^2} = f''(x) + \frac{h^2}{12}f^{(4)}(\xi_3), \quad \xi_3 \in (x-h, x+h).$$

The error is $\mathcal{O}(h^2)$.

Proof. We prove (2). Taylor-expand $f(x+h)$ and $f(x-h)$:

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4), \\ f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4). \end{aligned}$$

Subtracting:

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{3}f'''(x) + \mathcal{O}(h^5).$$

Dividing by $2h$: $\frac{f(x+h)-f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \mathcal{O}(h^4)$. By the intermediate value theorem, $f'''(x) = f'''(\xi_2)$ for some ξ_2 . $\square$

Warning

Rounding vs. truncation error trade-off. For machine epsilon ε , the total error in the centered difference is approximately

$$E(h) \approx \frac{h^2}{6}|f'''(x)| + \frac{\varepsilon|f(x)|}{h}.$$

The optimal step size is $h^* \approx \varepsilon^{1/3}$, giving error $\approx \varepsilon^{2/3}$.

Example 7.3. For $f(x) = \sin(x)$ at $x = 1$, the exact derivative is $\cos(1) \approx 0.5403023$. Errors:

h	Forward $\mathcal{O}(h)$	Centered $\mathcal{O}(h^2)$
10^{-1}	4.6×10^{-2}	1.4×10^{-3}
10^{-2}	4.6×10^{-3}	1.4×10^{-5}
10^{-4}	4.6×10^{-5}	1.4×10^{-9}
10^{-8}	4.6×10^{-9}	6.1×10^{-11}
10^{-12}	5.3×10^{-5}	4.2×10^{-5}

Note the loss of accuracy for very small h due to round-off.

7.2.2 Higher-order formulas

Proposition 7.4 (Fourth-order approximation of f').

$$f'(x) = \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h} + \mathcal{O}(h^4).$$

7.3 Numerical Integration (Quadrature)

7.3.1 The trapezoidal rule

Definition 7.5 (Simple trapezoidal rule).

$$\int_a^b f(x) dx \approx T(f) = \frac{b-a}{2} [f(a) + f(b)].$$

Theorem 7.6 (Trapezoidal rule error). If $f \in C^2([a, b])$,

$$\int_a^b f(x) dx - T(f) = -\frac{(b-a)^3}{12} f''(\xi), \quad \xi \in (a, b).$$

Proof. Let $I = [a, b]$ with $h = b - a$. The interpolation polynomial of degree 1 at a and b is

$$p_1(x) = f(a) \frac{b-x}{h} + f(b) \frac{x-a}{h}.$$

The interpolation error is $f(x) - p_1(x) = \frac{f''(\eta(x))}{2}(x-a)(x-b)$ for some $\eta(x) \in (a, b)$. Integrating:

$$\int_a^b f(x) dx - \frac{h}{2} [f(a) + f(b)] = \int_a^b \frac{f''(\eta(x))}{2} (x-a)(x-b) dx.$$

Since $(x-a)(x-b) \leq 0$ on $[a, b]$, by the mean value theorem for integrals:

$$= \frac{f''(\xi)}{2} \int_a^b (x-a)(x-b) dx = \frac{f''(\xi)}{2} \cdot \left(-\frac{h^3}{6}\right) = -\frac{h^3}{12} f''(\xi). \quad \square$$

Definition 7.7 (Composite trapezoidal rule). Dividing $[a, b]$ into n equal sub-intervals of width $h = \frac{b-a}{n}$ with $x_k = a + kh$:

$$T_n(f) = \frac{h}{2} \left[f(x_0) + 2 \sum_{k=1}^{n-1} f(x_k) + f(x_n) \right].$$

Theorem 7.8 (Composite trapezoidal error). If $f \in C^2([a, b])$,

$$\int_a^b f(x) dx - T_n(f) = -\frac{(b-a)h^2}{12} f''(\xi) = \mathcal{O}(h^2), \quad \xi \in (a, b).$$

7.3.2 Simpson's rule

Definition 7.9 (Simple Simpson's rule).

$$\int_a^b f(x) dx \approx S(f) = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right].$$

Theorem 7.10 (Simpson's rule error). If $f \in C^4([a, b])$,

$$\int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi), \quad \xi \in (a, b).$$

Simpson's rule is exact for polynomials of degree ≤ 3 (one degree of superconvergence).

Definition 7.11 (Composite Simpson's rule). With n even, $h = \frac{b-a}{n}$, and $x_k = a + kh$:

$$S_n(f) = \frac{h}{3} \left[f(x_0) + 4 \sum_{\substack{k=1 \\ k \text{ odd}}}^{n-1} f(x_k) + 2 \sum_{\substack{k=2 \\ k \text{ even}}}^{n-2} f(x_k) + f(x_n) \right].$$

Theorem 7.12 (Composite Simpson error). If $f \in C^4([a, b])$,

$$\int_a^b f(x) dx - S_n(f) = -\frac{(b-a)h^4}{180} f^{(4)}(\xi) = \mathcal{O}(h^4), \quad \xi \in (a, b).$$

Remark 7.13. Comparison of convergence orders:

Rule	Degree of exactness	Order
Trapezoidal	1	$\mathcal{O}(h^2)$
Simpson	3	$\mathcal{O}(h^4)$

7.4 Richardson Extrapolation

Definition 7.14 (Richardson extrapolation). If an approximation $A(h)$ satisfies $A(h) = A_0 + c_1 h^p + \mathcal{O}(h^{p+q})$, then

$$A^*(h) = \frac{2^p A(h/2) - A(h)}{2^p - 1}$$

is a higher-order approximation: $A^*(h) = A_0 + \mathcal{O}(h^{p+q})$.

Example 7.15 (Romberg integration). Applying Richardson extrapolation to the composite trapezoidal rule (where the error has only even powers of h):

- $T(h)$ and $T(h/2)$ combine to give Simpson's rule: $S = \frac{4T(h/2) - T(h)}{3}$;
- Further extrapolation yields Boole's rule and higher-order formulas.

The Romberg table:

$$\begin{array}{cccc} T_{1,1} & & & \\ T_{2,1} & T_{2,2} & & \\ T_{3,1} & T_{3,2} & T_{3,3} & \\ T_{4,1} & T_{4,2} & T_{4,3} & T_{4,4} \end{array}$$

where $T_{i,1} = T_{2^{i-1}}$ (composite trapezoidal with 2^{i-1} sub-intervals) and

$$T_{i,j} = \frac{4^{j-1} T_{i,j-1} - T_{i-1,j-1}}{4^{j-1} - 1}.$$

7.5 Convergence Study: $\int_0^1 e^{-x^2} dx$

Example 7.16 (Convergence table). We compute $I = \int_0^1 e^{-x^2} dx \approx 0.746824132812427$ using the composite trapezoidal and Simpson rules with increasing n :

n	$ I - T_n $	ratio	$ I - S_n $	ratio
2	2.83×10^{-2}	—	5.12×10^{-4}	—
4	7.18×10^{-3}	3.94	3.27×10^{-5}	15.7
8	1.80×10^{-3}	3.99	2.06×10^{-6}	15.9
16	4.51×10^{-4}	4.00	1.29×10^{-7}	16.0
32	1.13×10^{-4}	4.00	8.06×10^{-9}	16.0
64	2.82×10^{-5}	4.00	5.04×10^{-10}	16.0
128	7.05×10^{-6}	4.00	3.15×10^{-11}	16.0

The ratio $|E_n|/|E_{2n}|$ confirms $\mathcal{O}(h^2)$ for trapezoidal (ratio $\rightarrow 4$) and $\mathcal{O}(h^4)$ for Simpson (ratio $\rightarrow 16$).

7.6 Implementations

Python — Numerical integration

```
import numpy as np

def composite_trapezoidal(f, a, b, n):
    """Composite trapezoidal rule with n sub-intervals."""
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    return h * (0.5*f(x[0]) + np.sum(f(x[1:-1])) + 0.5*f(x[-1]))

def composite_simpson(f, a, b, n):
    """Composite Simpson's rule (n must be even)."""
    assert n % 2 == 0, "n must be even for Simpson's rule"
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
```

```

    return (h/3) * (f(x[0]) + 4*np.sum(f(x[1:-1]))
                  + 2*np.sum(f(x[2:-2])) + f(x[-1]))

def romberg(f, a, b, max_order=5):
    """Romberg integration."""
    T = np.zeros((max_order, max_order))
    for i in range(max_order):
        n = 2**i
        T[i, 0] = composite_trapezoidal(f, a, b, n)
        for j in range(1, i + 1):
            T[i, j] = (4**j * T[i, j-1] - T[i-1, j-1]) / (4**j - 1)
    return T

# Convergence study
f = lambda x: np.exp(-x**2)
I_exact = 0.746824132812427

print(f"{'n':>5} {'|I-T_n|':>14} {'ratio':>8} "
      f"{'|I-S_n|':>14} {'ratio':>8}")
print("-" * 55)

err_T_prev, err_S_prev = None, None
for k in range(1, 8):
    n = 2**k
    T = composite_trapezoidal(f, 0, 1, n)
    S = composite_simpson(f, 0, 1, n)
    err_T = abs(I_exact - T)
    err_S = abs(I_exact - S)
    ratio_T = err_T_prev / err_T if err_T_prev else float('nan')
    ratio_S = err_S_prev / err_S if err_S_prev else float('nan')
    print(f"{'n':5d} {'err_T':14.2e} {'ratio_T':8.2f} "
          f"{'err_S':14.2e} {'ratio_S':8.2f}")
    err_T_prev, err_S_prev = err_T, err_S

```

Julia — Numerical integration

```

function composite_trapezoidal(f, a, b, n)
    h = (b - a) / n
    x = range(a, b, length=n+1)
    return h * (f(x[1])/2 + sum(f.(x[2:end-1])) + f(x[end])/2)
end

function composite_simpson(f, a, b, n)
    @assert iseven(n) "n must be even"
    h = (b - a) / n
    x = range(a, b, length=n+1)
    return (h/3) * (f(x[1]) + 4*sum(f.(x[2:2:end-1]))
                  + 2*sum(f.(x[3:2:end-2])) + f(x[end]))
end

```

```

# Convergence study
f(x) = exp(-x^2)
I_exact = 0.746824132812427

println("  n      |I-T_n|      ratio    |I-S_n|      ratio")
err_T_prev = err_S_prev = NaN
for k in 1:7
    n = 2^k
    T = composite_trapezoidal(f, 0.0, 1.0, n)
    S = composite_simpson(f, 0.0, 1.0, n)
    err_T = abs(I_exact - T)
    err_S = abs(I_exact - S)
    ratio_T = isnan(err_T_prev) ? NaN : err_T_prev / err_T
    ratio_S = isnan(err_S_prev) ? NaN : err_S_prev / err_S
    @printf("%4d  %12.2e  %7.2f  %12.2e  %7.2f\n",
           n, err_T, ratio_T, err_S, ratio_S)
    err_T_prev, err_S_prev = err_T, err_S
end

```

7.7 Exercises

Exercise 7.1 (★). Use Taylor expansions to derive the truncation error of the forward difference formula and verify that it is $\mathcal{O}(h)$.

Exercise 7.2 (★). Show that the trapezoidal rule is exact for polynomials of degree ≤ 1 .

Exercise 7.3 (★★). Show that Simpson's rule is exact for polynomials of degree ≤ 3 (not just ≤ 2). *Hint*: verify directly for $f(x) = x^3$ on $[a, b]$.

Exercise 7.4 (★★). For the centered difference approximation of $f'(x)$:

1. Express the total error (truncation + round-off) as a function of h .
2. Find the optimal h^* that minimizes the total error.
3. Compute h^* and the corresponding error for double precision ($\varepsilon \approx 10^{-16}$).

Exercise 7.5 (★★). Apply Romberg integration to $\int_0^1 \frac{4}{1+x^2} dx = \pi$. Build the Romberg table up to $T_{4,4}$ and compare with π .

Exercise 7.6 (★★★). Implement an adaptive Simpson rule:

1. Compute $S_1 = S(a, b)$ and $S_2 = S(a, m) + S(m, b)$ where $m = (a + b)/2$.
2. If $|S_2 - S_1| < 15\varepsilon$ (with ε a given tolerance), accept $S_2 + (S_2 - S_1)/15$.
3. Otherwise, recursively refine each half.

Test on $\int_0^1 \sqrt{x} dx$ (which has a singularity in the derivative at $x = 0$).

Summary of Key Formulas

Key Formulas

Finite differences:

$$\begin{aligned} f'(x) &\approx \frac{f(x+h) - f(x)}{h} & \mathcal{O}(h) \\ f'(x) &\approx \frac{f(x+h) - f(x-h)}{2h} & \mathcal{O}(h^2) \\ f''(x) &\approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} & \mathcal{O}(h^2) \end{aligned}$$

Optimal step (centered difference): $h^* \approx \varepsilon^{1/3}$, error $\approx \varepsilon^{2/3}$.

Composite trapezoidal rule:

$$T_n(f) = \frac{h}{2} \left[f(x_0) + 2 \sum_{k=1}^{n-1} f(x_k) + f(x_n) \right], \quad \text{error} = \mathcal{O}(h^2)$$

Composite Simpson's rule (n even):

$$S_n(f) = \frac{h}{3} \left[f(x_0) + 4 \sum_{k \text{ odd}} f(x_k) + 2 \sum_{k \text{ even}} f(x_k) + f(x_n) \right], \quad \text{error} = \mathcal{O}(h^4)$$

Richardson extrapolation:

$$A^*(h) = \frac{2^p A(h/2) - A(h)}{2^p - 1}$$

Romberg recurrence:

$$T_{i,j} = \frac{4^{j-1} T_{i,j-1} - T_{i-1,j-1}}{4^{j-1} - 1}$$

Chapter 8

Gaussian Quadrature

In 1814, Carl Friedrich Gauss publishes a memoir on approximate integration by interpolation at the roots of orthogonal polynomials. The result is striking: to integrate a polynomial of degree $2n - 1$ exactly, only n well-chosen evaluations suffice, against $2n$ for Simpson's rule. This optimal efficiency is no coincidence — it follows from a deep property of orthogonal polynomials that Gauss proves in a few pages. Today, Gauss quadrature is the spine of the finite-element method: every mechanical-simulation code (ANSYS, Abaqus, Code_Aster) evaluates billions of Gauss points per second. This chapter constructs Gauss-Legendre, Gauss-Chebyshev, and Gauss-Hermite formulas, proves their optimal order of precision, and presents the adaptive variants (Gauss-Kronrod) used in automatic quadrature.

8.1 Introduction

Newton–Cotes formulas (trapezoidal, Simpson) use *equally spaced* nodes. A natural question arises: *can we do better by choosing the nodes optimally?* Gaussian quadrature answers positively: with n nodes, one can integrate exactly all polynomials of degree up to $2n - 1$.

8.2 Fundamental Theorem of Gaussian Quadrature

Definition 8.1 (Quadrature rule). A quadrature rule on $[a, b]$ with weight function $w(x) > 0$ is an approximation

$$\int_a^b w(x) f(x) dx \approx Q_n(f) = \sum_{i=1}^n w_i f(x_i),$$

where $x_1, \dots, x_n$ are the **nodes** and $w_1, \dots, w_n$ are the **weights**.

Definition 8.2 (Degree of exactness). A quadrature rule has **degree of exactness** d if it integrates exactly all polynomials of degree $\leq d$ but not all polynomials of degree $d + 1$.

Theorem 8.3 (Maximum degree of exactness). *An n -point quadrature rule can have degree of exactness at most $2n - 1$. This maximum is achieved if and only if the nodes are the zeros of the degree- n orthogonal polynomial with respect to $w(x)$ on $[a, b]$.*

Theorem 8.4 (Gauss quadrature — existence and properties). *Let $\{p_k\}_{k \geq 0}$ be the sequence of orthogonal polynomials with respect to $\langle f, g \rangle = \int_a^b w(x) f(x) g(x) dx$. Then:*

1. *The n zeros of p_n are real, distinct, and lie in (a, b) .*
2. *Using these zeros $x_1, \dots, x_n$ as nodes, there exist unique weights $w_1, \dots, w_n$ such that Q_n has degree of exactness $2n - 1$.*
3. *All weights are positive: $w_i > 0$ for $i = 1, \dots, n$.*

Proof sketch. (1) Let $x_{i_1}, \dots, x_{i_k}$ ($k \leq n$) be the zeros of p_n in (a, b) where p_n changes sign. Define $q(x) = (x - x_{i_1}) \cdots (x - x_{i_k})$. Then $p_n(x) q(x) \geq 0$ on (a, b) (or ≤ 0), so

$$\int_a^b w(x) p_n(x) q(x) dx \neq 0.$$

But if $k < n$, then $\deg q < n$ and orthogonality of p_n to lower-degree polynomials gives $\int_a^b w(x) p_n(x) q(x) dx = 0$, a contradiction. Hence $k = n$: all n zeros are in (a, b) and p_n changes sign at each.

(2) Define the weights via Lagrange interpolation at the nodes:

$$w_i = \int_a^b w(x) \ell_i(x) dx, \quad \ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

For any polynomial f of degree $\leq 2n - 1$, write $f = p_n \cdot q + r$ with $\deg q, \deg r \leq n - 1$. Then:

$$\int_a^b w(x) f(x) dx = \underbrace{\int_a^b w(x) p_n(x) q(x) dx}_{=0 \text{ (orthogonality)}} + \int_a^b w(x) r(x) dx.$$

Since $\deg r \leq n - 1$, the interpolation quadrature integrates r exactly, and $f(x_i) = r(x_i)$ since $p_n(x_i) = 0$. Thus $Q_n(f) = Q_n(r) = \int_a^b w(x) r(x) dx = \int_a^b w(x) f(x) dx$.

(3) Set $f(x) = \ell_i(x)^2$ (degree $2n - 2 \leq 2n - 1$). Then $Q_n(f) = w_i > 0$ since $\int_a^b w(x) \ell_i(x)^2 dx > 0$. $\square$

8.3 Gauss–Legendre Quadrature

Definition 8.5 (Gauss–Legendre quadrature). For $w(x) = 1$ on $[-1, 1]$, the Gauss–Legendre quadrature uses the zeros of the Legendre polynomial $P_n(x)$:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i).$$

Example 8.6 (Gauss–Legendre nodes and weights).

n	Nodes x_i	Weights w_i	Exactness
1	0	2	$d = 1$
2	$\pm 1/\sqrt{3}$	1	$d = 3$
3	$0, \pm\sqrt{3/5}$	$8/9, 5/9$	$d = 5$
4	$\pm\sqrt{\frac{3-2\sqrt{6/5}}{7}}, \pm\sqrt{\frac{3+2\sqrt{6/5}}{7}}$	$\frac{18+\sqrt{30}}{36}, \frac{18-\sqrt{30}}{36}$	$d = 7$
5	$0, \pm\frac{1}{3}\sqrt{5-2\sqrt{10/7}}, \pm\frac{1}{3}\sqrt{5+2\sqrt{10/7}}$	$\frac{128}{225}, \frac{322+13\sqrt{70}}{900}, \frac{322-13\sqrt{70}}{900}$	$d = 9$

8.3.1 Change of interval

Proposition 8.7 (Interval transformation). *To apply Gauss–Legendre quadrature on a general interval $[a, b]$, use the linear transformation $x = \frac{b-a}{2}t + \frac{a+b}{2}$:*

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right) dt \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right).$$

8.4 Error Formula

Theorem 8.8 (Gauss quadrature error). *If $f \in C^{2n}([a, b])$, the error of the n -point Gauss quadrature is*

$$E_n(f) = \int_a^b w(x) f(x) dx - \sum_{i=1}^n w_i f(x_i) = \frac{f^{(2n)}(\xi)}{(2n)!} \int_a^b w(x) \left[\prod_{i=1}^n (x - x_i) \right]^2 dx$$

for some $\xi \in (a, b)$.

Corollary 8.9 (Gauss–Legendre error). *For n -point Gauss–Legendre quadrature on $[-1, 1]$:*

$$E_n(f) = \frac{2^{2n+1}(n!)^4}{(2n+1)[(2n)!]^3} f^{(2n)}(\xi), \quad \xi \in (-1, 1).$$

8.5 Other Gaussian Quadrature Rules

8.5.1 Gauss–Laguerre

Definition 8.10 (Gauss–Laguerre quadrature). For integrals of the form $\int_0^\infty e^{-x} f(x) dx$, the nodes are the zeros of the Laguerre polynomial $L_n(x)$:

$$\int_0^\infty e^{-x} f(x) dx \approx \sum_{i=1}^n w_i f(x_i).$$

Example 8.11 (Gauss–Laguerre nodes and weights ($n = 3$)).

i	x_i	w_i
1	0.415775	0.711093
2	2.294280	0.278518
3	6.289945	0.010389

8.5.2 Gauss–Hermite

Definition 8.12 (Gauss–Hermite quadrature). For integrals of the form $\int_{-\infty}^{\infty} e^{-x^2} f(x) dx$, the nodes are the zeros of the Hermite polynomial $H_n(x)$:

$$\int_{-\infty}^{\infty} e^{-x^2} f(x) dx \approx \sum_{i=1}^n w_i f(x_i).$$

Example 8.13 (Gauss–Hermite nodes and weights ($n = 3$)).

i	x_i	w_i
1	$-\sqrt{3/2}$	$\sqrt{\pi}/6$
2	0	$2\sqrt{\pi}/3$
3	$\sqrt{3/2}$	$\sqrt{\pi}/6$

Remark 8.14. Summary of Gaussian quadrature variants:

Type	Interval	$w(x)$	Polynomials
Gauss–Legendre	$[-1, 1]$	1	Legendre P_n
Gauss–Chebyshev	$[-1, 1]$	$(1 - x^2)^{-1/2}$	Chebyshev T_n
Gauss–Laguerre	$[0, \infty)$	e^{-x}	Laguerre L_n
Gauss–Hermite	$(-\infty, \infty)$	e^{-x^2}	Hermite H_n

8.6 Composite Gauss Quadrature

Method

To improve accuracy without using a large number of nodes per element, subdivide $[a, b]$ into panels and apply Gauss quadrature on each panel. With m panels of equal width $h = (b - a)/m$ and n -point Gauss quadrature on each:

$$\int_a^b f(x) dx \approx \sum_{k=1}^m \frac{h}{2} \sum_{i=1}^n w_i f\left(\frac{h}{2} t_i + \frac{x_{k-1} + x_k}{2}\right).$$

The overall error is $\mathcal{O}(h^{2n})$.

8.7 Implementations

Python — Gauss–Legendre quadrature

```
import numpy as np
from numpy.polynomial.legendre import leggauss

def gauss_legendre(f, a, b, n):
    """n-point Gauss-Legendre quadrature on [a, b]."""
```



```

nodes, weights = leggauss(n)
# Transform from [-1, 1] to [a, b]
t = 0.5 * (b - a) * nodes + 0.5 * (a + b)
return 0.5 * (b - a) * np.sum(weights * f(t))

# Test: integral of exp(-x^2) from 0 to 1
f = lambda x: np.exp(-x**2)
I_exact = 0.746824132812427

print(f"{'n':>4} {'Approximation':>18} {'Error':>14}")
print("-" * 40)
for n in [2, 3, 4, 5, 8, 10, 15, 20]:
    approx = gauss_legendre(f, 0, 1, n)
    err = abs(I_exact - approx)
    print(f"{'n':4d} {'approx':18.15f} {'err':14.2e}")

# Composite Gauss-Legendre
def composite_gauss(f, a, b, n_nodes, n_panels):
    """Composite Gauss-Legendre quadrature."""
    h = (b - a) / n_panels
    result = 0.0
    for k in range(n_panels):
        ak = a + k * h
        bk = ak + h
        result += gauss_legendre(f, ak, bk, n_nodes)
    return result

# Compare convergence
print("\nComposite Gauss-Legendre (3-point per panel):")
for m in [1, 2, 4, 8, 16]:
    approx = composite_gauss(f, 0, 1, 3, m)
    err = abs(I_exact - approx)
    print(f"  panels = {m:3d}, error = {err:.2e}")

```

Julia — Gauss–Legendre quadrature with FastGaussQuadrature

```

using FastGaussQuadrature

function gauss_legendre(f, a, b, n)
    nodes, weights = gausslegendre(n)
    # Transform from [-1, 1] to [a, b]
    t = @. 0.5 * (b - a) * nodes + 0.5 * (a + b)
    return 0.5 * (b - a) * sum(weights .* f.(t))
end

# Test
f(x) = exp(-x^2)
I_exact = 0.746824132812427

```

```
println("    n      Approximation      Error")
println("-" ^ 45)
for n in [2, 3, 4, 5, 8, 10, 15, 20]
    approx = gauss_legendre(f, 0.0, 1.0, n)
    err = abs(I_exact - approx)
    @printf("%4d  %18.15f  %14.2e\n", n, approx, err)
end

# Gauss-Laguerre
println("\nGauss-Laguerre for integral of x^2 * exp(-x):")
for n in [3, 5, 10]
    nodes, weights = gausslaguerre(n)
    approx = sum(weights .* nodes.^2) # exact = 2
    @printf("  n = %2d, approx = %.15f, error = %.2e\n",
           n, approx, abs(2.0 - approx))
end

# Gauss-Hermite
println("\nGauss-Hermite for integral of x^4 * exp(-x^2):")
for n in [3, 5, 10]
    nodes, weights = gausshermite(n)
    approx = sum(weights .* nodes.^4) # exact = 3*sqrt(pi)/4
    exact = 3*sqrt(pi)/4
    @printf("  n = %2d, approx = %.15f, error = %.2e\n",
           n, approx, abs(exact - approx))
end
```

8.8 Exercises

Exercise 8.1 (*). Verify that the 2-point Gauss–Legendre rule ($x_{1,2} = \pm 1/\sqrt{3}$, $w_{1,2} = 1$) is exact for $f(x) = x^3$ on $[-1, 1]$.

Exercise 8.2 (*). Use 3-point Gauss–Legendre quadrature to approximate $\int_0^2 e^x dx$. Compare with the exact value $e^2 - 1$.

Exercise 8.3 (**). Derive the 2-point Gauss–Legendre rule from scratch by requiring exactness for $f(x) = 1, x, x^2, x^3$. Solve the resulting nonlinear system for x_1, x_2, w_1, w_2 .

Exercise 8.4 (**). 1. Implement composite Gauss–Legendre quadrature (3-point per panel).

2. Compute $\int_0^1 e^{-x^2} dx$ with $m = 1, 2, 4, 8, 16$ panels.

3. Verify numerically that the error is $\mathcal{O}(h^6)$ where $h = 1/m$.

Exercise 8.5 (**). Use Gauss–Laguerre quadrature to compute $\Gamma(s) = \int_0^\infty x^{s-1} e^{-x} dx$ for $s = 1.5, 2.5, 3.5$. Compare with the exact values $\Gamma(1.5) = \frac{\sqrt{\pi}}{2}$, $\Gamma(2.5) = \frac{3\sqrt{\pi}}{4}$, $\Gamma(3.5) = \frac{15\sqrt{\pi}}{8}$.

Exercise 8.6 (***). The Gauss quadrature nodes and weights can be computed from the eigenvalue problem of the tridiagonal Jacobi matrix.

1. For Gauss–Legendre, construct the symmetric tridiagonal matrix with $\alpha_k = 0$ and $\beta_k = k/\sqrt{4k^2 - 1}$.
2. Compute its eigenvalues (the nodes) and eigenvectors (to obtain the weights: $w_i = 2v_{1,i}^2$ where $v_{1,i}$ is the first component of the i -th eigenvector).
3. Compare with `numpy.polynomial.legendre.leggauss` for $n = 5, 10, 20$.

Summary of Key Formulas

Key Formulas

Gauss quadrature:

$$\int_a^b w(x) f(x) dx \approx \sum_{i=1}^n w_i f(x_i)$$

Nodes = zeros of the degree- n orthogonal polynomial; degree of exactness = $2n - 1$.

Gauss–Legendre on $[-1, 1]$ ($w(x) = 1$, Legendre polynomials):

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i), \quad w_i > 0, \quad \sum_{i=1}^n w_i = 2$$

Change of interval:

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right) dt$$

Error formula:

$$E_n(f) = \frac{f^{(2n)}(\xi)}{(2n)!} \int_a^b w(x) \left[\prod_{i=1}^n (x - x_i) \right]^2 dx$$

Summary table:

Gauss–Legendre	$[-1, 1]$	$w = 1$	Legendre P_n
Gauss–Laguerre	$[0, \infty)$	e^{-x}	Laguerre L_n
Gauss–Hermite	$(-\infty, \infty)$	e^{-x^2}	Hermite H_n

Chapter 9

Numerical Solution of Ordinary Differential Equations

In July 1969, the onboard computer of the Apollo 11 lunar module solves a system of ordinary differential equations in real time to compute the descent trajectory toward the Sea of Tranquility. The method — derived from the Runge-Kutta family, published by Carl Runge in 1895 and refined by Wilhelm Kutta in 1901 — has a single beautiful idea: replace the crude derivative of Euler’s method with a combination of slopes evaluated at intermediate points, achieving arbitrarily high convergence order. This chapter develops the explicit schemes (Euler, RK2, RK4), the implicit schemes (backward Euler, Crank-Nicolson, BDF) for stiff problems, analyzes their absolute stability, and presents adaptive step size — the default behind MATLAB `ode45` and `scipy solve_ivp`.

9.1 Introduction

Many phenomena in physics, biology, and engineering are modeled by ordinary differential equations (ODEs) that cannot be solved analytically. This chapter introduces the fundamental numerical methods for initial value problems.

9.2 The Cauchy Problem

Definition 9.1 (Initial value problem / Cauchy problem). Given $f : [t_0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$, find $y : [t_0, T] \rightarrow \mathbb{R}^d$ such that

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [t_0, T], \\ y(t_0) = y_0. \end{cases}$$

Theorem 9.2 (Picard–Lindelöf). *If f is continuous in t and Lipschitz continuous in y with Lipschitz constant L :*

$$\|f(t, y_1) - f(t, y_2)\| \leq L \|y_1 - y_2\| \quad \forall t \in [t_0, T], \quad y_1, y_2 \in \mathbb{R}^d,$$

then the Cauchy problem has a unique solution on $[t_0, T]$.

We discretize $[t_0, T]$ with a uniform grid: $t_n = t_0 + nh$, $h = (T - t_0)/N$, and seek approximations $y_n \approx y(t_n)$.

9.3 Euler's Methods

9.3.1 Explicit (forward) Euler method

Definition 9.3 (Explicit Euler method).

$$y_{n+1} = y_n + h f(t_n, y_n), \quad n = 0, 1, \dots, N-1.$$

Algorithm 6: Explicit Euler method

Input: f , t_0 , T , y_0 , N (number of steps)

Output: Arrays $t[0 : N]$, $y[0 : N]$

$h \leftarrow (T - t_0)/N$;

$t[0] \leftarrow t_0$;

$y[0] \leftarrow y_0$;

for $n = 0, 1, \dots, N-1$ **do**

$y[n+1] \leftarrow y[n] + h \cdot f(t[n], y[n]);$
 $t[n+1] \leftarrow t[n] + h$;

return t , y ;

Theorem 9.4 (Global error of explicit Euler). *If f is Lipschitz in y with constant L and $y \in C^2([t_0, T])$, then*

$$\max_{0 \leq n \leq N} \|y(t_n) - y_n\| \leq \frac{hM}{2L} (e^{L(T-t_0)} - 1),$$

where $M = \max_{t \in [t_0, T]} \|y''(t)\|$. The method is of **order 1**: the global error is $\mathcal{O}(h)$.

Proof. **Local truncation error.** Taylor-expanding:

$$y(t_{n+1}) = y(t_n) + h y'(t_n) + \frac{h^2}{2} y''(\xi_n) = y(t_n) + h f(t_n, y(t_n)) + \frac{h^2}{2} y''(\xi_n).$$

The local truncation error is $\tau_n = \frac{h^2}{2} y''(\xi_n)$, so $|\tau_n| \leq \frac{hM}{2}$.

Global error. Let $e_n = y(t_n) - y_n$. Then:

$$\begin{aligned} e_{n+1} &= y(t_{n+1}) - y_{n+1} \\ &= [y(t_n) + h f(t_n, y(t_n)) + h \tau_n] - [y_n + h f(t_n, y_n)] \\ &= e_n + h [f(t_n, y(t_n)) - f(t_n, y_n)] + h \tau_n. \end{aligned}$$

Using the Lipschitz condition:

$$\|e_{n+1}\| \leq (1 + hL) \|e_n\| + h \frac{hM}{2}.$$

Setting $\delta = \frac{hM}{2}$ and iterating with $e_0 = 0$:

$$\|e_n\| \leq \frac{\delta}{hL} [(1 + hL)^n - 1] \leq \frac{hM}{2L} [e^{nhL} - 1] \leq \frac{hM}{2L} [e^{L(T-t_0)} - 1],$$

using the inequality $(1 + hL)^n \leq e^{nhL}$. □

9.3.2 Implicit (backward) Euler method

Definition 9.5 (Implicit Euler method).

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}), \quad n = 0, 1, \dots, N-1.$$

This is an implicit equation for y_{n+1} and generally requires solving a nonlinear system at each step (e.g., by Newton's method).

Definition 9.6 (A-stability). A numerical method is **A-stable** if its stability region contains the entire left half of the complex plane: $\{z \in \mathbb{C} : \operatorname{Re}(z) \leq 0\} \subset \mathcal{S}$.

Theorem 9.7. *The implicit Euler method is A-stable.*

Proof. For the test equation $y' = \lambda y$ ($\lambda \in \mathbb{C}$), the implicit Euler gives:

$$y_{n+1} = y_n + h\lambda y_{n+1} \implies y_{n+1} = \frac{y_n}{1 - h\lambda}.$$

The amplification factor is $R(z) = \frac{1}{1-z}$ where $z = h\lambda$. Stability requires $|R(z)| \leq 1$, i.e., $|1-z| \geq 1$. For $z = x + iy$ with $x \leq 0$: $|1-z|^2 = (1-x)^2 + y^2 \geq 1$ since $1-x \geq 1$. Thus $|R(z)| \leq 1$ for all $\operatorname{Re}(z) \leq 0$. $\square$

9.4 Runge–Kutta Methods

Definition 9.8 (General s -stage Runge–Kutta method). An s -stage Runge–Kutta method is defined by

$$k_i = f\left(t_n + c_i h, y_n + h \sum_{j=1}^s a_{ij} k_j\right), \quad i = 1, \dots, s,$$

$$y_{n+1} = y_n + h \sum_{i=1}^s b_i k_i.$$

The method is **explicit** if $a_{ij} = 0$ for $j \geq i$.

Definition 9.9 (Butcher tableau). A Runge–Kutta method is compactly represented by its **Butcher tableau**:

$$\begin{array}{c|c} \mathbf{c} & \mathbf{A} \\ \hline & \mathbf{b}^\top \end{array} = \begin{array}{c|cccc} c_1 & a_{11} & a_{12} & \cdots & a_{1s} \\ c_2 & a_{21} & a_{22} & \cdots & a_{2s} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_s & a_{s1} & a_{s2} & \cdots & a_{ss} \\ \hline & b_1 & b_2 & \cdots & b_s \end{array}$$

with the consistency condition $c_i = \sum_{j=1}^s a_{ij}$.

9.4.1 Classical fourth-order Runge–Kutta (RK4)

Definition 9.10 (RK4). The classical RK4 method:

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + h/2, y_n + (h/2) k_1), \\ k_3 &= f(t_n + h/2, y_n + (h/2) k_2), \\ k_4 &= f(t_n + h, y_n + h k_3), \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Butcher tableau:

$$\begin{array}{c|ccc} 0 & & & \\ 1/2 & 1/2 & & \\ 1/2 & 0 & 1/2 & \\ 1 & 0 & 0 & 1 \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array}$$

Theorem 9.11 (Order of RK4). *The classical RK4 method has **order 4**: the global error is $\mathcal{O}(h^4)$ and the local truncation error is $\mathcal{O}(h^5)$.*

Remark 9.12. For explicit Runge–Kutta methods:

Method	Stages	Order
Euler	1	1
Heun (RK2)	2	2
Kutta (RK3)	3	3
Classic RK4	4	4

For $s \geq 5$ stages, the maximum achievable order is less than s (Butcher barrier).

9.5 Stability Analysis

Definition 9.13 (Test equation). The Dahlquist test equation is $y' = \lambda y$, $y(0) = 1$, with $\lambda \in \mathbb{C}$. Any one-step method applied to this equation produces

$$y_{n+1} = R(z) y_n, \quad z = h\lambda,$$

where $R(z)$ is the **amplification factor** (or stability function).

Definition 9.14 (Stability region). The **stability region** of a method is

$$\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \leq 1\}.$$

For the exact solution $|e^z| \leq 1$ when $\operatorname{Re}(z) \leq 0$, so a method is A-stable if $\mathcal{S} \supseteq \{z : \operatorname{Re}(z) \leq 0\}$.

Proposition 9.15 (Amplification factors).

<i>Method</i>	$R(z)$
<i>Explicit Euler</i>	$1 + z$
<i>Implicit Euler</i>	$\frac{1}{1 - z}$
<i>Trapezoidal (Crank–Nicolson)</i>	$\frac{1 + z/2}{1 - z/2}$
<i>RK4</i>	$1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$

Remark 9.16. The explicit Euler stability region is the disk $|1 + z| \leq 1$, centered at $(-1, 0)$ with radius 1. For the explicit Euler applied to $y' = \lambda y$ with $\lambda < 0$, stability requires $0 < h < 2/|\lambda|$.

9.5.1 Stability region plots

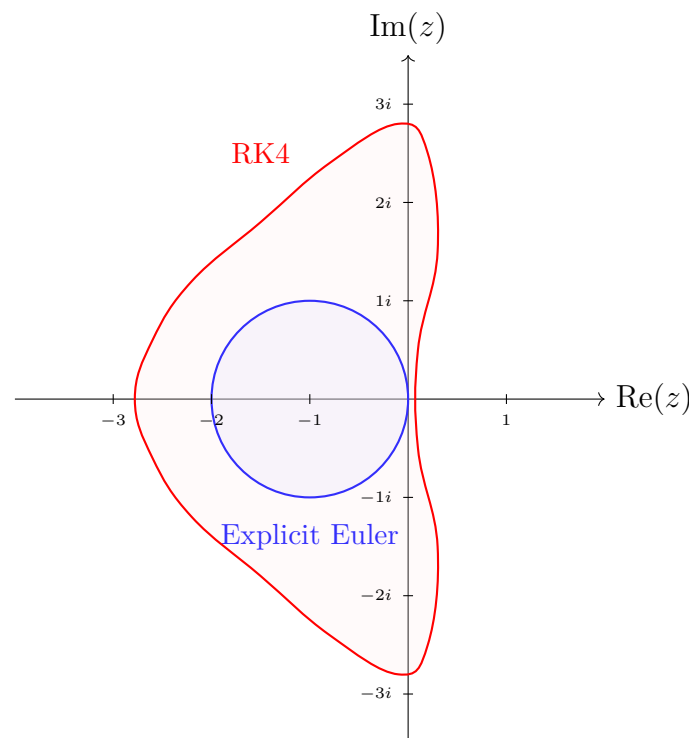


Figure 9.1: Stability regions of explicit Euler (blue) and RK4 (red) in the complex $z = h\lambda$ plane.

9.6 Convergence Study: $y' = -y$

Example 9.17 (Convergence table). We solve $y' = -y$, $y(0) = 1$ on $[0, 1]$ (exact: $y(1) = e^{-1}$):

N	$ e_N^{\text{Euler}} $	order	$ e_N^{\text{Heun}} $	order	$ e_N^{\text{RK4}} $	order
10	1.7×10^{-2}	—	4.1×10^{-4}	—	1.5×10^{-7}	—
20	8.5×10^{-3}	1.0	1.0×10^{-4}	2.0	9.3×10^{-9}	4.0
40	4.2×10^{-3}	1.0	2.6×10^{-5}	2.0	5.8×10^{-10}	4.0
80	2.1×10^{-3}	1.0	6.4×10^{-6}	2.0	3.6×10^{-11}	4.0
160	1.1×10^{-3}	1.0	1.6×10^{-6}	2.0	2.3×10^{-12}	4.0

9.7 Implementations

Python — ODE solvers

```
import numpy as np
import matplotlib.pyplot as plt

def euler_explicit(f, t0, T, y0, N):
    """Explicit Euler method."""
    h = (T - t0) / N
    t = np.linspace(t0, T, N + 1)
    y = np.zeros((N + 1, len(np.atleast_1d(y0))))
    y[0] = y0
    for n in range(N):
        y[n+1] = y[n] + h * f(t[n], y[n])
    return t, y

def euler_implicit(f, jac, t0, T, y0, N, tol=1e-12):
    """Implicit Euler method (Newton iteration)."""
    h = (T - t0) / N
    t = np.linspace(t0, T, N + 1)
    d = len(np.atleast_1d(y0))
    y = np.zeros((N + 1, d))
    y[0] = y0
    for n in range(N):
        # Newton iteration: G(u) = u - y[n] - h*f(t[n+1], u) = 0
        u = y[n].copy() # initial guess
        for _ in range(50):
            G = u - y[n] - h * f(t[n+1], u)
            dG = np.eye(d) - h * jac(t[n+1], u)
            delta = np.linalg.solve(dG, -G)
            u += delta
            if np.linalg.norm(delta) < tol:
                break
        y[n+1] = u
    return t, y

def rk4(f, t0, T, y0, N):
    """Classical RK4 method."""
    h = (T - t0) / N
    t = np.linspace(t0, T, N + 1)
```

```

y = np.zeros((N + 1, len(np.atleast_1d(y0))))
y[0] = y0
for n in range(N):
    k1 = f(t[n], y[n])
    k2 = f(t[n] + h/2, y[n] + h/2 * k1)
    k3 = f(t[n] + h/2, y[n] + h/2 * k2)
    k4 = f(t[n] + h, y[n] + h * k3)
    y[n+1] = y[n] + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
return t, y

# Test: y' = -y, y(0) = 1
f = lambda t, y: -y
y0 = np.array([1.0])
y_exact = lambda t: np.exp(-t)

print(f"{'N':>5} {'Euler':>14} {'order':>6} "
      f"{'RK4':>14} {'order':>6}")
print("-" * 50)
err_e_prev = err_r_prev = None
for N in [10, 20, 40, 80, 160, 320]:
    t_e, y_e = euler_explicit(f, 0, 1, y0, N)
    t_r, y_r = rk4(f, 0, 1, y0, N)
    err_e = abs(y_e[-1, 0] - y_exact(1.0))
    err_r = abs(y_r[-1, 0] - y_exact(1.0))
    ord_e = np.log2(err_e_prev/err_e) if err_e_prev else float('nan')
    ord_r = np.log2(err_r_prev/err_r) if err_r_prev else float('nan')
    print(f"{'N':5d} {'err_e':14.2e} {'ord_e':6.2f} "
          f"{'err_r':14.2e} {'ord_r':6.2f}")
    err_e_prev, err_r_prev = err_e, err_r

```

Julia — ODE solvers

```

function euler_explicit(f, t0, T, y0, N)
    h = (T - t0) / N
    t = range(t0, T, length=N+1)
    d = length(y0)
    y = zeros(d, N+1)
    y[:, 1] = y0
    for n in 1:N
        y[:, n+1] = y[:, n] + h * f(t[n], y[:, n])
    end
    return collect(t), y
end

function rk4(f, t0, T, y0, N)
    h = (T - t0) / N
    t = range(t0, T, length=N+1)
    d = length(y0)
    y = zeros(d, N+1)
    y[:, 1] = y0

```

```

    for n in 1:N
        k1 = f(t[n], y[:, n])
        k2 = f(t[n] + h/2, y[:, n] + h/2 * k1)
        k3 = f(t[n] + h/2, y[:, n] + h/2 * k2)
        k4 = f(t[n] + h, y[:, n] + h * k3)
        y[:, n+1] = y[:, n] + (h/6) * (k1 + 2k2 + 2k3 + k4)
    end
    return collect(t), y
end

# Test: y' = -y
f(t, y) = -y
y0 = [1.0]

for N in [10, 20, 40, 80, 160]
    t_e, y_e = euler_explicit(f, 0.0, 1.0, y0, N)
    t_r, y_r = rk4(f, 0.0, 1.0, y0, N)
    err_e = abs(y_e[1, end] - exp(-1.0))
    err_r = abs(y_r[1, end] - exp(-1.0))
    @printf("N = %4d: Euler = %.2e, RK4 = %.2e\n",
           N, err_e, err_r)
end

```

9.8 Exercises

Exercise 9.1 (*). Apply the explicit Euler method with $h = 0.25$ to solve $y' = -2y$, $y(0) = 1$ on $[0, 1]$. Compare with the exact solution $y(t) = e^{-2t}$.

Exercise 9.2 (*). For the test equation $y' = \lambda y$ with $\lambda = -10$:

1. Determine the maximum step size h for which the explicit Euler method is stable.
2. Show that the implicit Euler method is stable for any $h > 0$.

Exercise 9.3 (**). The midpoint method has Butcher tableau:

$$\begin{array}{c|cc} 0 & & \\ 1/2 & 1/2 & \\ \hline & 0 & 1 \end{array}$$

1. Write out the method explicitly.
2. Show it has order 2 by verifying the order conditions.
3. Determine its stability function $R(z)$ and plot the stability region.

Exercise 9.4 (**). Reproduce the convergence table of Example 9.17. Verify numerically the convergence orders of explicit Euler, Heun's method, and RK4.

Exercise 9.5 (★★). Consider the stiff system

$$y' = -1000(y - \sin t) + \cos t, \quad y(0) = 0.$$

1. Solve with explicit Euler for $h = 0.001$ and $h = 0.003$. What happens?
2. Solve with implicit Euler for $h = 0.01$ and $h = 0.1$.
3. Explain the results using stability analysis.

Exercise 9.6 (★★★ — Project: Lorenz system). The Lorenz system is

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = x(\rho - z) - y, \\ \dot{z} = xy - \beta z, \end{cases}$$

with the classical parameters $\sigma = 10$, $\rho = 28$, $\beta = 8/3$.

1. Implement the system and solve with RK4 for $t \in [0, 50]$ with $h = 0.01$.
2. Plot the trajectory in 3D (x – y – z phase space).
3. Investigate sensitivity to initial conditions: solve with $(x_0, y_0, z_0) = (1, 1, 1)$ and $(1.001, 1, 1)$. Plot $|x_1(t) - x_2(t)|$ and observe exponential divergence.
4. Estimate the largest Lyapunov exponent.

Exercise 9.7 (★★★). Implement an adaptive RK4-5 method (Dormand–Prince):

1. Use embedded RK pairs of order 4 and 5 to estimate the local error.
2. Adjust the step size: $h_{\text{new}} = h \cdot \min\left(2, \max\left(0.5, 0.9 \left(\frac{\text{tol}}{|e|}\right)^{1/5}\right)\right)$.
3. Test on $y' = -y^2$, $y(0) = 1$ (exact: $y = 1/(1+t)$) with tolerance 10^{-8} .

Summary of Key Formulas

Key Formulas

Explicit Euler:

$$y_{n+1} = y_n + h f(t_n, y_n), \quad \text{order 1, } R(z) = 1 + z$$

Implicit Euler:

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}), \quad \text{order 1, } R(z) = \frac{1}{1-z}, \quad \text{A-stable}$$

RK4:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad \text{order 4}$$

Global error bound (Euler):

$$\max_n \|y(t_n) - y_n\| \leq \frac{hM}{2L}(e^{L(T-t_0)} - 1)$$

Stability condition (explicit Euler): $z = h\lambda \in \{z : |1 + z| \leq 1\}$

A-stability: stability region $\supseteq \{z \in \mathbb{C} : \operatorname{Re}(z) \leq 0\}$

Butcher tableau:

c	A
	b

Chapter 10

Numerical Computation of Eigenvalues and Eigenvectors

In 1998, two Stanford doctoral students — Sergey Brin and Larry Page — model the World Wide Web as a directed graph with several million pages, and try to rank each page by importance. Their algorithm, PageRank, reduces mathematically to computing the dominant eigenvector of a giant stochastic matrix. They use the power method, published by Richard von Mises in 1929: pick an initial vector, multiply by the matrix, normalize, repeat; the sequence converges to the dominant eigenvector. That is how Google began. This chapter covers the modern iterative methods for eigenvalue problems: power iteration, inverse iteration, Rayleigh quotient iteration, the QR algorithm (cornerstone of LAPACK), and the Krylov methods (Lanczos, Arnoldi) that dominate computation for sparse matrices.

10.1 Introduction

The eigenvalue problem—finding λ and $\mathbf{v} \neq \mathbf{0}$ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ —arises throughout science and engineering: vibration analysis, stability of dynamical systems, principal component analysis, Google’s PageRank, and quantum mechanics, among many others. Since the eigenvalues are roots of the characteristic polynomial $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$, and there is no closed-form formula for roots of polynomials of degree ≥ 5 (Abel–Ruffini theorem), all general eigenvalue algorithms are necessarily *iterative*.

10.2 Spectral Properties

Definition 10.1 (Spectrum). The **spectrum** of $\mathbf{A} \in \mathbb{R}^{n \times n}$ (or $\mathbb{C}^{n \times n}$) is the set of its eigenvalues:

$$\sigma(\mathbf{A}) = \{\lambda \in \mathbb{C} : \det(\mathbf{A} - \lambda\mathbf{I}) = 0\}.$$

The **spectral radius** is $\rho(\mathbf{A}) = \max_{\lambda \in \sigma(\mathbf{A})} |\lambda|$.

Proposition 10.2 (Basic spectral properties). *Let $\mathbf{A} \in \mathbb{C}^{n \times n}$ with eigenvalues $\lambda_1, \dots, \lambda_n$ (counted with multiplicity).*

1. $\sum_{i=1}^n \lambda_i = \text{tr}(\mathbf{A}), \quad \prod_{i=1}^n \lambda_i = \det(\mathbf{A}).$
2. If $\mathbf{A}$ is real and $\lambda \in \sigma(\mathbf{A})$, then $\bar{\lambda} \in \sigma(\mathbf{A})$.
3. If $\mathbf{A}$ is symmetric (Hermitian), all eigenvalues are real.
4. If $\mathbf{A}$ is symmetric positive definite, all eigenvalues are positive.
5. $\sigma(\mathbf{A}^{-1}) = \{1/\lambda : \lambda \in \sigma(\mathbf{A})\}$ when $\mathbf{A}$ is invertible.
6. $\sigma(\mathbf{A} - \sigma \mathbf{I}) = \{\lambda - \sigma : \lambda \in \sigma(\mathbf{A})\}$ (spectral shift).

10.3 Gerschgorin's Theorem

Theorem 10.3 (Gerschgorin circle theorem). *Every eigenvalue of $\mathbf{A} \in \mathbb{C}^{n \times n}$ lies in at least one of the **Gerschgorin discs**:*

$$D_i = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i\}, \quad R_i = \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|.$$

That is, $\sigma(\mathbf{A}) \subset \bigcup_{i=1}^n D_i$.

Moreover, if a union of k discs is disjoint from the remaining $n - k$ discs, it contains exactly k eigenvalues (counted with multiplicity).

Proof. Let λ be an eigenvalue with eigenvector $\mathbf{v}$, and let i be the index of the component of largest modulus: $|v_i| = \|\mathbf{v}\|_\infty$. From $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, the i -th equation gives:

$$\sum_{j=1}^n a_{ij} v_j = \lambda v_i \implies (\lambda - a_{ii}) v_i = \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} v_j.$$

Taking absolute values:

$$|\lambda - a_{ii}| |v_i| \leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| |v_j| \leq \left(\sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \right) |v_i| = R_i |v_i|.$$

Since $|v_i| > 0$, we get $|\lambda - a_{ii}| \leq R_i$, i.e., $\lambda \in D_i$. □

Example 10.4 (Gerschgorin discs). For the matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 1 & 0 \\ 0.5 & 3 & 0.5 \\ 0 & 1 & 6 \end{pmatrix},$$

the Gerschgorin discs are:

- D_1 : center 4, radius 1, i.e., $[3, 5]$;
- D_2 : center 3, radius 1, i.e., $[2, 4]$;
- D_3 : center 6, radius 1, i.e., $[5, 7]$.

The actual eigenvalues are approximately 2.59, 4.25, 6.16, all within the union of discs. Since D_3 is disjoint from $D_1 \cup D_2$, it contains exactly one eigenvalue.

10.3.1 Gerschgorin discs visualization

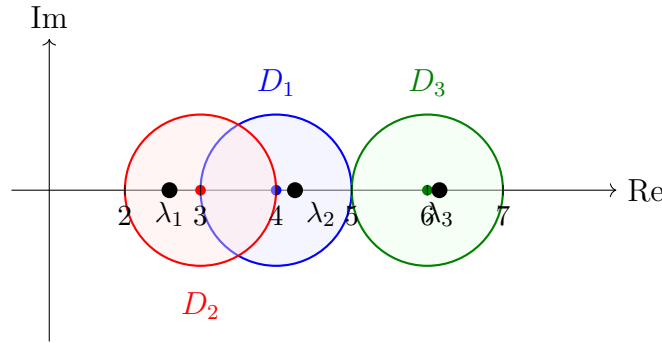


Figure 10.1: Gerschgorin discs for the matrix $\mathbf{A}$. Black dots indicate the actual eigenvalues.

10.4 Power Iteration

Definition 10.5 (Power iteration). The **power iteration** (or power method) computes the dominant eigenvalue (largest in modulus) and its eigenvector.

Algorithm 7: Power iteration

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, initial vector $\mathbf{v}^{(0)}$, tolerance ε , max iterations $k_{\max}$

Output: Dominant eigenvalue λ , eigenvector $\mathbf{v}$

$\mathbf{v} \leftarrow \mathbf{v}^{(0)} / \|\mathbf{v}^{(0)}\|_2$;

for $k = 1, 2, \dots, k_{\max}$ **do**

$\mathbf{w} \leftarrow \mathbf{A} \mathbf{v}$;

$\lambda \leftarrow \mathbf{v}^\top \mathbf{w}$ // Rayleigh quotient

$\mathbf{v} \leftarrow \mathbf{w} / \|\mathbf{w}\|_2$;

if $\|\mathbf{A} \mathbf{v} - \lambda \mathbf{v}\|_2 < \varepsilon$ **then**

return $\lambda, \mathbf{v}$;

return $\lambda, \mathbf{v}$;

Theorem 10.6 (Convergence of power iteration). *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ have eigenvalues $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ with corresponding eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ forming a basis. If $\mathbf{v}^{(0)}$ has a nonzero component in the direction of $\mathbf{v}_1$, then:*

1. The Rayleigh quotient converges to λ_1 :

$$|\lambda^{(k)} - \lambda_1| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right).$$

2. The iterates $\mathbf{v}^{(k)}$ converge to $\pm \mathbf{v}_1$ (up to normalization):

$$\|\mathbf{v}^{(k)} - (\pm \mathbf{v}_1)\| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right).$$

The rate of convergence is $r = |\lambda_2/\lambda_1| < 1$.

Proof. Write the initial vector as $\mathbf{v}^{(0)} = \sum_{i=1}^n c_i \mathbf{v}_i$ with $c_1 \neq 0$. After k iterations (ignoring normalization):

$$\mathbf{A}^k \mathbf{v}^{(0)} = \sum_{i=1}^n c_i \lambda_i^k \mathbf{v}_i = \lambda_1^k \left[c_1 \mathbf{v}_1 + \sum_{i=2}^n c_i \left(\frac{\lambda_i}{\lambda_1}\right)^k \mathbf{v}_i \right].$$

Since $|\lambda_i/\lambda_1| < 1$ for $i \geq 2$, the sum converges to $c_1 \lambda_1^k \mathbf{v}_1$, so the normalized iterate converges to $\pm \mathbf{v}_1$.

For the Rayleigh quotient, $\lambda^{(k)} = (\mathbf{v}^{(k)})^\top \mathbf{A} \mathbf{v}^{(k)}$. Since the eigenvector error is $\mathcal{O}(r^k)$, the eigenvalue error is $\mathcal{O}(r^{2k})$ (the Rayleigh quotient has a quadratic stationary property for symmetric matrices). $\square$

Remark 10.7. The convergence can be very slow when $|\lambda_2/\lambda_1|$ is close to 1. This motivates the use of shifted methods and more sophisticated algorithms.

10.5 Inverse Iteration with Shift

Definition 10.8 (Inverse iteration with shift). To find the eigenvalue closest to a given shift σ , we apply power iteration to $(\mathbf{A} - \sigma \mathbf{I})^{-1}$, whose dominant eigenvalue is $1/(\lambda_j - \sigma)$ where λ_j is the eigenvalue of $\mathbf{A}$ closest to σ .

Algorithm 8: Inverse iteration with shift

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, shift σ , initial $\mathbf{v}^{(0)}$, tolerance ε , $k_{\max}$

Output: Eigenvalue λ closest to σ , eigenvector $\mathbf{v}$

$\mathbf{v} \leftarrow \mathbf{v}^{(0)} / \|\mathbf{v}^{(0)}\|_2$;

Compute LU factorization of $\mathbf{A} - \sigma \mathbf{I}$;

for $k = 1, 2, \dots, k_{\max}$ **do**

 Solve $(\mathbf{A} - \sigma \mathbf{I}) \mathbf{w} = \mathbf{v}$ // using LU factors

$\lambda \leftarrow \mathbf{v}^\top \mathbf{A} \mathbf{v}$ // Rayleigh quotient

$\mathbf{v} \leftarrow \mathbf{w} / \|\mathbf{w}\|_2$;

if $\|\mathbf{A} \mathbf{v} - \lambda \mathbf{v}\|_2 < \varepsilon$ **then**

return $\lambda, \mathbf{v}$;

return $\lambda, \mathbf{v}$;

Proposition 10.9. *Inverse iteration with shift σ converges at rate $\left| \frac{\lambda_j - \sigma}{\lambda_k - \sigma} \right|$, where λ_j is the nearest eigenvalue to σ and λ_k is the second nearest. With a good estimate of σ , convergence can be very fast.*

Remark 10.10 (Rayleigh quotient iteration). If the shift σ is updated at each step to the current Rayleigh quotient, we obtain the **Rayleigh quotient iteration**, which converges cubically for symmetric matrices.

10.6 The QR Algorithm

10.6.1 Basic QR algorithm

Definition 10.11 (QR algorithm). Starting from $\mathbf{A}_0 = \mathbf{A}$, the basic QR algorithm iterates:

$$\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k \quad (\text{QR factorization}), \quad \mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k.$$

Proposition 10.12 (Properties of the QR iteration). 1. All matrices $\mathbf{A}_k$ are similar: $\mathbf{A}_{k+1} = \mathbf{Q}_k^\top \mathbf{A}_k \mathbf{Q}_k$.

2. If $\mathbf{A}$ has eigenvalues $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > 0$, then $\mathbf{A}_k$ converges to an upper triangular matrix with the eigenvalues on the diagonal.

3. The convergence rate of the sub-diagonal entry $(i+1, i)$ is $\mathcal{O}(|\lambda_{i+1}/\lambda_i|^k)$.

10.6.2 QR algorithm with Hessenberg reduction

Definition 10.13 (Hessenberg form). A matrix is in **upper Hessenberg form** if $a_{ij} = 0$ for $i > j + 1$:

$$\mathbf{H} = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{n,n-1} & h_{nn} \end{pmatrix}.$$

Theorem 10.14 (Hessenberg reduction). Any matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ can be reduced to upper Hessenberg form by an orthogonal similarity transformation: $\mathbf{H} = \mathbf{Q}^\top \mathbf{A} \mathbf{Q}$, using Householder reflections, in $\mathcal{O}(n^3)$ operations.

Method

The practical QR algorithm proceeds in two phases:

1. **Reduction:** Transform $\mathbf{A}$ to Hessenberg form $\mathbf{H}$ (cost: $\mathcal{O}(n^3)$, done once).
2. **Iteration:** Apply the QR iteration to $\mathbf{H}$, which preserves Hessenberg form. Each QR step on a Hessenberg matrix costs only $\mathcal{O}(n^2)$.

For symmetric matrices, Hessenberg form is tridiagonal, and each step costs $\mathcal{O}(n)$.

10.6.3 QR algorithm with shifts

Definition 10.15 (Shifted QR iteration).

$$\mathbf{A}_k - \sigma_k \mathbf{I} = \mathbf{Q}_k \mathbf{R}_k, \quad \mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k + \sigma_k \mathbf{I}.$$

The shift σ_k is chosen to accelerate convergence.

Definition 10.16 (Wilkinson shift). The **Wilkinson shift** is the eigenvalue of the bottom-right 2×2 submatrix

$$\begin{pmatrix} a_{n-1,n-1} & a_{n-1,n} \\ a_{n,n-1} & a_{nn} \end{pmatrix}$$

that is closer to a_{nn} . Explicitly:

$$\sigma = a_{nn} + \delta - \text{sign}(\delta) \sqrt{\delta^2 + a_{n,n-1}^2}, \quad \delta = \frac{a_{n-1,n-1} - a_{nn}}{2}.$$

Theorem 10.17. *With the Wilkinson shift, the QR algorithm converges cubically for symmetric matrices and at least quadratically in general.*

Remark 10.18. The complete modern QR algorithm (as implemented in LAPACK) combines:

- Hessenberg reduction;
- Implicit double shifts (Francis QR step) to handle complex conjugate eigenvalue pairs in real arithmetic;
- Deflation when a sub-diagonal element becomes negligible;
- Exceptional shifts for difficult cases.

The total cost is $\mathcal{O}(n^3)$ for all eigenvalues.

10.7 Singular Value Decomposition (SVD)

Theorem 10.19 (Singular Value Decomposition). *Every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (with $m \geq n$) admits a factorization*

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top,$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is orthogonal (columns: left singular vectors),
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is orthogonal (columns: right singular vectors),
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is “diagonal” with $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ (the **singular values**).

Proposition 10.20 (Properties of the SVD). 1. $\text{rank}(\mathbf{A}) = \text{number of nonzero singular values}.$

2. $\|\mathbf{A}\|_2 = \sigma_1$, $\|\mathbf{A}\|_F = \sqrt{\sigma_1^2 + \cdots + \sigma_n^2}$.
3. $\text{cond}_2(\mathbf{A}) = \sigma_1/\sigma_n$ (when $\mathbf{A}$ is square and invertible).
4. The singular values of $\mathbf{A}$ are the square roots of the eigenvalues of $\mathbf{A}^\top \mathbf{A}$.
5. $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ (outer product form), where $r = \text{rank}(\mathbf{A})$.

Theorem 10.21 (Eckart–Young theorem). *The best rank- k approximation to $\mathbf{A}$ (in both the 2-norm and the Frobenius norm) is*

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top,$$

where $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, $\mathbf{V}_k$ are the truncated SVD. The approximation errors are:

$$\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}, \quad \|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_n^2}.$$

10.8 Convergence Study

Example 10.22 (Power iteration convergence). For the matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix},$$

the eigenvalues are $\lambda_1 \approx 4.732$, $\lambda_2 \approx 3.000$, $\lambda_3 \approx 1.268$. The ratio $r = |\lambda_2/\lambda_1| \approx 0.634$.

k	$\lambda^{(k)}$	$ \lambda^{(k)} - \lambda_1 $	$ \lambda_2/\lambda_1 ^k$	$ \lambda_2/\lambda_1 ^{2k}$
1	4.5000	2.32×10^{-1}	6.34×10^{-1}	4.02×10^{-1}
2	4.7143	1.77×10^{-2}	4.02×10^{-1}	1.61×10^{-1}
3	4.7302	1.76×10^{-3}	2.55×10^{-1}	6.48×10^{-2}
5	4.7321	1.38×10^{-5}	1.02×10^{-1}	1.04×10^{-2}
10	4.7321	3.32×10^{-11}	1.05×10^{-2}	1.09×10^{-4}

The eigenvalue error decreases at rate $\mathcal{O}(r^{2k})$ (Rayleigh quotient).

10.9 Implementations

Python — Eigenvalue algorithms

```
import numpy as np
import matplotlib.pyplot as plt

def power_iteration(A, v0=None, tol=1e-12, maxiter=1000):
```

```

"""Power iteration for the dominant eigenvalue."""
n = A.shape[0]
v = v0 if v0 is not None else np.random.randn(n)
v = v / np.linalg.norm(v)
history = []
for k in range(maxiter):
    w = A @ v
    lam = v @ w # Rayleigh quotient
    v = w / np.linalg.norm(w)
    history.append(lam)
    if np.linalg.norm(A @ v - lam * v) < tol:
        break
return lam, v, history

def inverse_iteration(A, sigma, v0=None, tol=1e-12, maxiter=1000):
    """Inverse iteration with shift sigma."""
    n = A.shape[0]
    v = v0 if v0 is not None else np.random.randn(n)
    v = v / np.linalg.norm(v)
    B = A - sigma * np.eye(n)
    lu_piv = None
    try:
        from scipy.linalg import lu_factor, lu_solve
        lu_piv = lu_factor(B)
    except ImportError:
        pass
    history = []
    for k in range(maxiter):
        if lu_piv is not None:
            w = lu_solve(lu_piv, v)
        else:
            w = np.linalg.solve(B, v)
        lam = v @ (A @ v) # Rayleigh quotient
        v = w / np.linalg.norm(w)
        history.append(lam)
        if np.linalg.norm(A @ v - lam * v) < tol:
            break
    return lam, v, history

def qr_algorithm(A, maxiter=200, tol=1e-12):
    """Basic QR algorithm (without shifts)."""
    n = A.shape[0]
    Ak = A.copy().astype(float)
    for k in range(maxiter):
        Q, R = np.linalg.qr(Ak)
        Ak = R @ Q
        # Check convergence: sub-diagonal elements
        off = np.sum(np.abs(np.tril(Ak, -1)))
        if off < tol:
            break
    return np.diag(Ak), Ak

```

```

# Example
A = np.array([[4, 1, 0],
              [1, 3, 1],
              [0, 1, 2]], dtype=float)

print("=== Power iteration ===")
lam, v, hist = power_iteration(A)
print(f"Dominant eigenvalue: {lam:.10f}")
print(f"Eigenvector: {v}")

print("\n=== Inverse iteration (sigma=1.0) ===")
lam2, v2, hist2 = inverse_iteration(A, sigma=1.0)
print(f"Nearest eigenvalue to 1.0: {lam2:.10f}")

print("\n=== QR algorithm ===")
eigenvalues, _ = qr_algorithm(A)
print(f"All eigenvalues: {np.sort(eigenvalues)[::-1]}")

print("\n=== NumPy reference ===")
eigvals = np.linalg.eigvalsh(A)
print(f"numpy.linalg.eigvalsh: {np.sort(eigvals)[::-1]}")

# SVD example
print("\n=== SVD ===")
M = np.array([[3, 2, 2],
              [2, 3, -2]], dtype=float)
U, s, Vt = np.linalg.svd(M)
print(f"Singular values: {s}")
print(f"Rank: {np.sum(s > 1e-10)}")
print(f"||M||_2 = {s[0]:.6f}")
print(f"cond_2(M) = {s[0]/s[-1]:.6f}")

# Low-rank approximation
k = 1
Mk = U[:, :k] @ np.diag(s[:k]) @ Vt[:, k, :]
print(f"Best rank-{k} approximation error: {s[k]:.6f}")

# Gerschgorin discs plot
fig, ax = plt.subplots(1, 1, figsize=(8, 4))
for i in range(A.shape[0]):
    center = A[i, i]
    radius = sum(abs(A[i, j]) for j in range(A.shape[1]) if j != i)
    circle = plt.Circle((center, 0), radius, fill=True,
                        alpha=0.2, edgecolor='blue', linewidth=2)
    ax.add_patch(circle)
    ax.plot(center, 0, 'b.', markersize=8)

eigvals_exact = np.linalg.eigvalsh(A)
ax.plot(eigvals_exact, np.zeros_like(eigvals_exact), 'k*',
        markersize=12, label='Eigenvalues')

```

```
ax.set_xlim(0, 7)
ax.set_ylim(-2.5, 2.5)
ax.set_aspect('equal')
ax.axhline(y=0, color='gray', linewidth=0.5)
ax.set_xlabel('Re')
ax.set_ylabel('Im')
ax.set_title('Gerschgorin discs')
ax.legend()
plt.tight_layout()
plt.savefig('gerschgorin.pdf')
plt.show()
```

Julia — Eigenvalue algorithms

```
using LinearAlgebra, Printf

function power_iteration(A; v0=nothing, tol=1e-12, maxiter=1000)
    n = size(A, 1)
    v = isnothing(v0) ? randn(n) : copy(v0)
    v ./= norm(v)
    lam = 0.0
    for k in 1:maxiter
        w = A * v
        lam = dot(v, w)
        v = w / norm(w)
        if norm(A * v - lam * v) < tol
            @printf("Converged in %d iterations\n", k)
            break
        end
    end
    return lam, v
end

function inverse_iteration(A, sigma; tol=1e-12, maxiter=1000)
    n = size(A, 1)
    v = randn(n)
    v ./= norm(v)
    F = lu(A - sigma * I)
    lam = 0.0
    for k in 1:maxiter
        w = F \ v
        lam = dot(v, A * v)
        v = w / norm(w)
        if norm(A * v - lam * v) < tol
            @printf("Converged in %d iterations\n", k)
            break
        end
    end
    return lam, v
end
```



```

end

# Example
A = [4.0 1.0 0.0;
     1.0 3.0 1.0;
     0.0 1.0 2.0]

println("=== Power iteration ===")
lam, v = power_iteration(A)
@printf("Dominant eigenvalue: %.10f\n", lam)

println("\n=== Inverse iteration (sigma=1.0) ===")
lam2, v2 = inverse_iteration(A, 1.0)
@printf("Nearest eigenvalue to 1.0: %.10f\n", lam2)

println("\n=== Built-in eigvals ===")
println("Eigenvalues: ", eigvals(Symmetric(A)))

println("\n=== SVD ===")
M = [3.0 2.0 2.0; 2.0 3.0 -2.0]
F = svd(M)
println("Singular values: ", F.S)

```

10.10 Exercises

Exercise 10.1 (★). For the matrix

$$\mathbf{A} = \begin{pmatrix} 5 & 1 & 1 \\ 0 & 4 & 1 \\ 1 & 0 & -1 \end{pmatrix},$$

1. Determine the three Gerschgorin discs.
2. Sketch them in the complex plane.
3. Deduce bounds on the eigenvalues. In particular, show that one eigenvalue is negative.

Exercise 10.2 (★). Apply 3 iterations of power iteration to

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{v}^{(0)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Compare the Rayleigh quotient at each step with the exact dominant eigenvalue.

Exercise 10.3 (★★). For the matrix in Exercise 10.2:

1. Apply inverse iteration with shift $\sigma = 1.5$ to find the smallest eigenvalue.

2. Compare the convergence rate with unshifted power iteration.

Exercise 10.4 (**). Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$.

1. Compute $\mathbf{A}^\top \mathbf{A}$ and its eigenvalues.
2. Deduce the singular values of $\mathbf{A}$.
3. Verify with `numpy.linalg.svd`.
4. Compute the best rank-1 approximation and its error.

Exercise 10.5 (**). Implement the basic QR algorithm and apply it to the matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 4 \end{pmatrix}.$$

1. Track $\max_{i>j} |a_{ij}^{(k)}|$ (sub-diagonal elements) as a function of iteration k .
2. How many iterations are needed for convergence to 10^{-12} ?
3. Add the Wilkinson shift and compare.

Exercise 10.6 (***). (Project: Image compression via SVD)

1. Load a grayscale image as a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$.
2. Compute its SVD: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.
3. Plot the singular values (in log scale). How fast do they decay?
4. Reconstruct the image using rank- k approximations for $k = 1, 5, 10, 20, 50, 100$.
5. For each k , compute the compression ratio: $\frac{k(m+n+1)}{mn}$.
6. Plot the relative error $\|\mathbf{A} - \mathbf{A}_k\|_F / \|\mathbf{A}\|_F$ as a function of k .

Exercise 10.7 (***). Implement Hessenberg reduction using Householder reflections:

1. For a general $n \times n$ matrix, compute $\mathbf{H} = \mathbf{Q}^\top \mathbf{A} \mathbf{Q}$.
2. Verify that $\mathbf{H}$ is upper Hessenberg and $\sigma(\mathbf{H}) = \sigma(\mathbf{A})$.
3. Implement the QR iteration on the Hessenberg matrix and compare the cost with the basic QR algorithm.

Summary of Key Formulas

Key Formulas

Gerschgorin:

$$\sigma(\mathbf{A}) \subset \bigcup_{i=1}^n D_i, \quad D_i = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i\}, \quad R_i = \sum_{j \neq i} |a_{ij}|$$

Power iteration: converges to λ_1 at rate $|\lambda_2/\lambda_1|^k$ (eigenvector) or $|\lambda_2/\lambda_1|^{2k}$ (Rayleigh quotient).

Inverse iteration with shift σ : converges to the eigenvalue nearest σ , rate $\left| \frac{\lambda_j - \sigma}{\lambda_k - \sigma} \right|^k$.

QR algorithm:

$$\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k, \quad \mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k \rightarrow \text{upper triangular (eigenvalues on diagonal)}$$

Wilkinson shift: $\sigma = a_{nn} + \delta - \text{sign}(\delta) \sqrt{\delta^2 + a_{n,n-1}^2}$, $\delta = (a_{n-1,n-1} - a_{nn})/2$.

SVD: $\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^\top$, $\|\mathbf{A}\|_2 = \sigma_1$, $\text{cond}_2(\mathbf{A}) = \sigma_1/\sigma_n$.

Eckart–Young: Best rank- k approximation is truncated SVD, error = σ_{k+1} (2-norm).

Formula Sheet

.1 Vector and Matrix Norms

$$\|x\|_1 = \sum |x_i|, \quad \|x\|_2 = \sqrt{\sum x_i^2}, \quad \|x\|_\infty = \max |x_i| \quad (1)$$

$$\|A\|_1 = \max_j \sum_i |a_{ij}|, \quad \|A\|_\infty = \max_i \sum_j |a_{ij}|, \quad \|A\|_2 = \sigma_{\max}(A) \quad (2)$$

.2 Errors

$$\text{Absolute error : } |x - \tilde{x}| \quad (3)$$

$$\text{Relative error : } \frac{|x - \tilde{x}|}{|x|} \quad (4)$$

$$\text{Condition number : } \text{cond}(A) = \|A\| \cdot \|A^{-1}\| \quad (5)$$

.3 Quadrature Rules

$$\text{Trapezoidal : } \frac{h}{2}[f(a) + 2 \sum f(x_i) + f(b)], \quad E = \mathcal{O}(h^2) \quad (6)$$

$$\text{Simpson : } \frac{h}{3}[f(a) + 4 \sum f_{\text{odd}} + 2 \sum f_{\text{even}} + f(b)], \quad E = \mathcal{O}(h^4) \quad (7)$$

.4 ODE Methods

$$\text{Explicit Euler : } y_{n+1} = y_n + hf(t_n, y_n), \quad \mathcal{O}(h) \quad (8)$$

$$\text{RK4 : } y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad \mathcal{O}(h^4) \quad (9)$$

Bibliography

- [1] QUARTERONI, A., Sacco, R. & Saleri, F. (2007). *Numerical Mathematics*. 2nd ed., Springer.
- [2] SÜLI, E. & Mayers, D. (2003). *An Introduction to Numerical Analysis*. Cambridge University Press.
- [3] TREFETHEN, L.N. & Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
- [4] HIGHAM, N.J. (2002). *Accuracy and Stability of Numerical Algorithms*. 2nd ed., SIAM.
- [5] DEMAILLY, J.-P. (2006). *Analyse Numérique et Équations Différentielles*. EDP Sciences.